Behavior-Guided Multimodal Policy Search: Leveraging Vision-Language Models for Reinforcement Learning

Vu Gia Nam, Nguyen Tran Minh Nhat, Nguyen Viet Anh, Pham Vu Tuan Dat, Long Doan, Loc Nguyen, Huynh Thi Thanh Binh

Abstract

Large language models (LLMs) have emerged as promising black-box optimizers for reinforcement learning (RL) policy search, yet existing approaches operate in a purely textual feedback loop, relying on scalar rewards, rankings, or natural-language summaries that reveal whether a policy succeeds or fails but provide little insight into why it fails. To address this limitation, we introduce Behavior-guided Multimodal Policy Search (BMPS), a framework that augments LLM-driven optimization with structured visual diagnostics. First, BMPS constructs a local behavioral landscape by perturbing policy parameters, extracting representative trajectory frames, and using vision-language models (VLMs) to identify failure modes, spatial patterns, and behavioral deviations. Then, we integrate this multimodal feedback with reward histories into the LLM prompt to guide policy search. Across discrete and continuous control tasks, BMPS consistently outperforms policy-gradient baselines, prior LLM-driven optimization methods, and evolutionary approaches, with the largest gains in environments with sparse or weak informative rewards. Taken together, these results show that grounding policy search in interpretable behavioral feedback makes LLM-based optimization more effective and diagnosis-aware.

Keywords

Reinforcement Learning, Policy Search, Large Language Models, Vision-Language Models, Multimodal Feedback, Black-Box Optimization

1 Introduction

Reinforcement Learning (RL) [31] addresses a fundamental problem in machine learning: learning decision-making policies through interaction with an environment. RL has achieved remarkable results across complex domains, from Atari games [18, 19] to Go, Chess, and Shogi [29, 30], and Dota 2 [24]. Traditional methods such as PPO [28] and TRPO [27] rely on scalar reward signals, which offer only sparse, evaluative feedback with no semantic structure, limiting interpretability and exploration efficiency. The rise of large language models (LLMs) has led to their increasing integration into RL pipelines, where they are primarily used as supporting components.

LLMs have recently been explored in RL both as auxiliary modules, such as reward shapers or sources of prior knowledge [35, 37], and as more general black-box optimizers [4, 36]. Nevertheless, when applied to RL policy search, existing LLM-based optimizers still rely almost exclusively on scalar reward signals. This form of feedback indicates whether a policy performs well or poorly, but it does not explain which concrete behaviors caused success or failure. As a result, the model receives only weak supervision for policy refinement: it may observe that one candidate is better than another, yet still lack the behavioral evidence needed to infer how the policy should change. This limitation is especially severe in environments with sparse, delayed, or deceptive rewards, where return values alone may obscure the underlying reasons for poor performance. Consequently, the LLM cannot fully leverage its strengths in semantic reasoning, comparison, and diagnosis, and policy search then becomes more like trial and error based on rewards, rather than a process of learning from clear behavioral insights.

To address these limitations, we propose Behavior-guided Multimodal Policy Search (BMPS), a novel framework for reinforcement learning policy optimization with multimodal diagnostic feedback. BMPS advances LLM-based policy search in two key ways. First, BMPS treats the LLM as a direct optimizer over policy parameters, allowing policy updates to be generated through iterative language-based reasoning rather than relying solely on conventional gradient-based optimization. Second, and most importantly, BMPS introduces visual feedback-driven behavioral analysis into the optimization loop. By leveraging vision-language models (VLMs) to examine representative trajectory frames beyond scalar rewards alone, BMPS enables targeted and diagnosis-aware policy refinement. This transforms LLM-based policy search from a largely reward-driven trial-and-error process into a more grounded and interpretable optimization procedure. In our evaluation, BMPS has higher mean returns than prior LLM-based optimizers on 12 of 13 tasks and ties on Maze, while comparisons with policy-gradient and evolutionary baselines vary by task. In particular, BMPS falls below the leading baselines on Hopper, Navigation, and Inverted Double Pendulum.

In summary, our contributions are as follows:

- **Behavior-guided multimodal policy optimization.** We propose a framework that augments LLM-based policy search with structured visual feedback through a novel neighborhood behavioral sampling mechanism. Specifically, BMPS perturbs policy parameters via Poisson sampling around anchor policies (current, best, worst), performs rollouts, and leverages vision-language models (VLMs) to analyze trajectory frames in parallel. This process constructs a local behavioral landscape that captures how small parameter variations affect observed behaviors, enabling diagnosis-aware optimization beyond scalar reward signals.
- **Regret analysis of behavior-guided search.** We show $O(\sqrt{T})$ Bayesian regret when the LLM approximates posterior sampling and bound the simple regret of the returned policy under a weak condition on the proposals. Both results are tied to behavioral feedback through a measurable feedback effect, whose expectation splits into the decision value of the descriptions and the proposal regret of the LLM.
- **Empirical evaluation across RL benchmarks.** We evaluate BMPS on 13 discrete- and continuous-control tasks,

comparing against policy-gradient baselines, prior LLM-driven optimization methods, and evolutionary approaches. BMPS achieves the best average rank and higher mean returns than the prior LLM-based methods on 12 tasks, tying on Maze.

2 Related Work

Classical reinforcement learning optimizes policies from scalar returns using gradient-based methods such as TRPO and PPO [27, 28] or derivative-free search [17], achieving strong results across sequential decision-making problems [31]. However, scalar feedback provides limited information about underlying behaviors and failure modes.

To enrich supervision, prior work incorporates language into RL. Early approaches used linguistic signals for exploration, transfer, and interpretability [6, 10, 22], but were limited in expressiveness and scalability [15]. With modern LLMs, free-form text enables broader integration into RL pipelines [35, 39]. Most existing methods, however, use language as auxiliary guidance rather than as the optimization mechanism itself. A prominent direction is reward synthesis, where LLMs translate human intent into reward functions or evaluative signals [11, 13, 16, 37]. For example, DrEureka designs rewards and environment variations but relies on PPO for optimization [16], while language model predictive control converts feedback into executable control programs [13]. These approaches require collapsing semantic feedback into scalar objectives, which can introduce ambiguity and reward misspecification [9].Other work integrates language more directly into decision-making, such as guiding actions [7], modifying value updates [34], or coordinating modules [14], but still does not treat LLMs as optimizers over policy parameters.

Our work instead studies LLMs as policy optimizers, closely related to in-context RL and algorithm distillation [12, 40], contextual bandit formulations [21], and the view of LLMs as black-box optimizers [4, 36], as well as recent policy refinement approaches [1, 5, 38]. In particular, Prompted Policy Search (ProPS) [38] treats the LLM as a direct policy optimizer that proposes policy parameters from parameter–reward histories, while ProPS+ further incorporates semantic task information such as environment descriptions, goals, and expert hints. These methods are especially relevant to our setting and directly inspire BMPS. However, both still rely primarily on textual and scalar feedback, without grounding optimization in trajectory-level behavioral analysis. BMPS extends this line of work by integrating VLM-based visual diagnostics with neighborhood policy sampling to construct a local behavioral landscape, enabling multimodal behavior-guided policy refinement beyond reward-only prompting.

3 Methodology

3.1 Problem Formulation

We consider an episodic environment with state space $\mathcal{S}$, action space $\mathcal{A}$, reward function r, and horizon H. A policy π_θ with parameters $\theta \in \Theta$ induces a distribution p_θ over trajectories τ, and the goal is to maximize the expected return

$$J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\big[R(\tau)\big], \qquad R(\tau) = \sum_{h=0}^{H-1} r(s_h, a_h), \tag{1}$$

where an episode may terminate before the horizon. The optimizer treats J as a black box: it observes neither gradients nor a model of the dynamics. A policy proposed by the optimizer is evaluated by the Monte Carlo estimate

$$\widehat{J}(\theta) = \frac{1}{L}\sum_{i=1}^{L} R(\tau_i), \qquad \tau_1, \ldots, \tau_L \overset{\text{i.i.d.}}{\sim} p_\theta, \tag{2}$$

with $L = 20$ rollouts in our experiments.

Policy classes. For continuous state spaces, we use linear policies $\pi_\theta(s) = W^\top s + b$, where $\theta \in \mathbb{R}^d$ stacks the entries of W and b; for discrete action sets, the action with the largest linear score is executed. Each parameter is restricted to $[-6, 6]$ with one decimal place, which gives the grid $\Theta = \{-6, -5.9, \ldots, 6\}^d$ with 121^d elements. Despite their simplicity, linear policies are competitive when combined with random-search optimization [17]. For discrete state spaces, we use deterministic tabular policies $\pi_\theta(s) = \theta_s$ with $\theta_s \in \mathcal{A}$, so that $\Theta = \mathcal{A}^{|\mathcal{S}|}$ has $|\mathcal{A}|^{|\mathcal{S}|}$ elements. Both classes are finite; the regret bounds in Section 4 use finiteness only through $\log|\Theta|$ and do not rely on the grid structure.

Prompted policy search. At iteration t, the optimizer holds the history $\mathcal{H}_{t-1} = \{(\theta_k, \widehat{J}(\theta_k))\}$ of all evaluated policies, including W randomly sampled warm-up policies, and a textual task description c. A reward-only prompted optimizer such as ProPS [38] draws the next policy from $P_{\text{LLM}}(\cdot \mid \mathcal{H}_{t-1}, c)$, the distribution that the LLM and its decoding procedure induce on policies once the response is parsed into a parameter vector. We call any such prompt-induced distribution a *proposal distribution*. Except in Theorem 4.4, which assumes that it approximates posterior sampling, we make no assumption about its form; in particular, the LLM is not assumed to perform Bayesian inference. BMPS enriches the prompt with a behavioral landscape, described next.

3.2 Behavior-guided Multimodal Policy Search

BMPS augments prompted policy search with a *behavioral landscape*: a set of rollouts around selected anchor policies, each paired with a VLM description of what the agent did. Each iteration consists of the steps below (Figure 1). Figure 2 illustrates how this evidence enters proposals in a logged search run.

At each iteration t, the algorithm maintains a history H_{t-1} of all previously evaluated policies and their associated rewards, along with two behavioral anchors: the best-performing policy $\theta_{\text{best}} = \arg\max_{\theta_k} r_k$ and the worst-performing policy $\theta_{\text{worst}} = \arg\min_{\theta_k} r_k$ observed so far. The current policy θ_{curr} is initialized to the final randomly sampled policy from the warm-up phase and subsequently updated to θ_t at the end of each iteration. The current anchor connects diagnostics to the latest proposal, the best retains an empirically successful reference, and the worst supplies examples of failure. Together, they provide contrasting behaviors within the same policy class, although their selection by estimated returns need not reflect the true ordering.

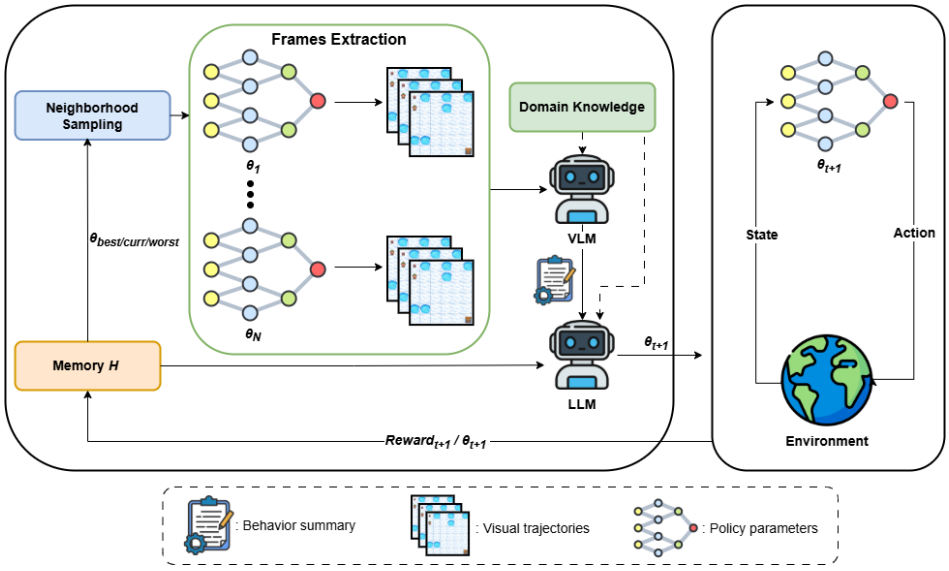


Figure 1: Overview of Behavior-guided Multimodal Policy Search (BMPS). At each iteration, the current policy is evaluated through environment rollouts, representative trajectory frames are extracted, and a vision-language model generates structured behavioral feedback. This multimodal feedback, together with reward statistics and policy history, is provided to the large language model to propose improved policy parameters for the next round of search.

Neighborhood Sampling. To build a local picture of the *behavioral landscape*, we sample N neighbor policies via Poisson perturbation around each of the three reference points $\theta \in \{\theta_{\text{best}}, \theta_{\text{worst}}, \theta_{\text{curr}}\}$. Given a reference parameter vector θ, each neighbor $\tilde{\theta}$ is generated by perturbing every dimension d by $\delta_d = s_d \cdot \sigma_d \cdot \epsilon$, where $s_d \sim \text{Poisson}(\lambda)$ is a non-negative integer step count, $\sigma_d \sim \text{Uniform}(\{-1, +1\})$ is a random sign, and ϵ is a fixed step size. The Poisson distribution is chosen deliberately: it concentrates mass near zero to favor small, local perturbations while its tail permits occasional larger jumps, naturally balancing local exploitation with broader exploration around each reference point. The integer step count fits the discrete parameter representation and allows some coordinates to remain unchanged. Since nearby parameters can still yield different behaviors, we use the rollouts as comparative evidence rather than interpret the perturbations as gradient estimates; useful exploration is not guaranteed.

Multimodal Analysis. The distinguishing feature of our approach is the use of a VLM to analyze the visual content of the collected trajectories, using either a joint VLM call or per-trajectory VLM summaries. For each trajectory, we sample one frame every 50 environment steps until the trajectory terminates. Given the sampled frames from the current policy rollout, the best and worst anchor trajectories, and their respective neighborhood rollouts, the VLM produces a structured behavioral summary ψ_t. This summary captures qualitative differences in agent behavior across trajectories-such as spatial positioning, motion patterns, object interactions, and failure signatures-that are difficult or impossible to encode through reward alone. By grounding the analysis in raw visual observations, the model can identify behaviorally meaningful distinctions that scalar metrics obscure. Separating visual diagnosis from policy proposal provides an inspectable text interface that the optimizer can combine with numerical evidence. Per-trajectory summaries also allow diagnostic calls to run in parallel. This compression can miss brief events or misinterpret observations, so it is a trade-off rather than an assumed advantage over direct image input.

Language-guided Policy Proposal. Conditioned on the history H_{t-1} and the multimodal analysis ψ_t, the LLM proposes a new candidate policy θ_t. This proposal step functions as an informed, semantics-aware mutation: rather than sampling blindly from a fixed perturbation kernel, the LLM uses its understanding of the behavioral summaries to suggest a policy that steers toward the characteristics of successful trajectories while explicitly departing from the patterns associated with failure. The history H_{t-1} provides additional context about which regions of the policy space have been explored and what reward signals they yielded, allowing the LLM to use both past successes and failures when generating the next proposal. Associating parameter changes with both descriptions and returns gives the LLM concrete alternatives to compare, while retaining history provides a record of earlier trials beyond the current neighborhood. These comparisons may inform a proposal, but a plausible behavioral explanation does not itself establish an improvement.

Evaluation and History Update. The proposed policy θ_t is then evaluated by collecting its estimated return $r_t = \hat{J}(\theta_t)$ from L fresh rollouts, and the history is extended to $H_t = H_{t-1} \cup \{(\theta_t, r_t)\}$. The anchors θ_{best} and θ_{worst} are updated accordingly if the new evaluation yields a more extreme reward in either direction. Single-rollout neighborhoods limit the cost of collecting diagnostic comparisons, whereas fresh evaluation rollouts reduce the influence of an isolated

favorable outcome on the recorded return. Neighborhood rollouts are not added to the selection history, and the returned policy is chosen from the evaluated proposals and warm-up policies, with finite-sample selection error still possible.

Together, these steps combine visual diagnostics, numerical returns, and search history. Behavioral similarity need not correspond to proximity in parameter space, so the landscape supplies comparisons of observed outcomes rather than a smooth local model of return. Section 4 distinguishes the information available in this feedback from the error incurred when the optimizer uses it.

4 Theoretical Analysis

Setup. We assume that returns lie in a known range $[r_{\min}, r_{\max}]$, let $B = r_{\max} - r_{\min}$, and use the normalized return $g(\tau) = (R(\tau) - r_{\min})/B \in [0,1]$ and its expectation $j(\theta) = \mathbb{E}_{\tau\sim p_\theta}[g(\tau)]$, so that $J = r_{\min} + Bj$. Let $C \subseteq \Theta$ be a finite set that contains every policy the parsing step can produce. At iteration t, the LLM receives the prompt $Z_t = (X_t, \Psi_t)$. The numerical context $X_t = (\mathcal{H}_{t-1}, \mathcal{N}_t, c)$ contains the history, the parameters and single-rollout returns $\mathcal{N}_t$ of all anchors and neighbors, and the task description, and Ψ_t contains the VLM descriptions of these rollouts. Since an LLM defines a conditional distribution over its outputs [2], BMPS draws its proposal from $Q_t = P_{\mathrm{LLM}}(\cdot \mid X_t, \Psi_t)$, whereas a numerical-only proposal at the same context would follow $P_t = P_{\mathrm{LLM}}(\cdot \mid X_t)$. Both distributions can be sampled, but their probabilities are never computed. Prompts are finite strings and take countably many values. We write $Z_{1:t} = (Z_1, \ldots, Z_t)$; the record Z_{t+1} contains θ_t and its estimated return, and Z_{T+1} is the record after the last evaluation.

Assumption 4.1 (Prompt-only proposals). *Conditionally on Z_t, the proposal θ_t is independent of the environment and of all other quantities generated before the LLM is queried at iteration t, and its distribution is Q_t.*

This holds by construction when the LLM sees only the prompt, its decoding randomness is drawn afresh, and the served model does not change during the run. The *cumulative regret* of the proposals is $\sum_{t=1}^{T}(j(\theta^*) - j(\theta_t))$, where θ^* is a best policy, and the *simple regret* is $j(\theta^*) - j(\widehat{\theta}_T)$ for the policy $\widehat{\theta}_T$ that the search returns.

4.1 Analytical Model and the Feedback Effect

Assumption 4.2 (Analytical task model). *The environment instance ξ is drawn from an arbitrary distribution, $j_\xi(\theta) \in [0,1]$ is its normalized expected return, and BMPS runs under ξ, so that ξ, the prompts, and the proposals have a joint distribution. We let θ^* be a C-valued random variable with $j_\xi(\theta^*) = \max_{\theta\in C} j_\xi(\theta)$, with ties broken by a fixed rule.*

This assumption is a modeling device, not a restriction on the system. The distribution of ξ need not be correctly specified, the LLM is assumed neither to know it nor to perform Bayesian inference, and the results below hold for every such distribution. The *Bayesian regret* of the proposals is

$$\mathrm{BR}_T = \mathbb{E}\Big[\sum_{t=1}^{T}\big(j_\xi(\theta^*) - j_\xi(\theta_t)\big)\Big]. \tag{3}$$

The feedback effect. Behavioral feedback enters the analysis through one measurable quantity. For an environment ξ and a context $z = (x, \psi)$, let $\mu_\xi^Q(z) = \mathbb{E}_{\theta\sim Q(\cdot|z)}[j_\xi(\theta)]$ and $\mu_\xi^P(z) = \mathbb{E}_{\theta\sim P(\cdot|x)}[j_\xi(\theta)]$, where $Q(\cdot \mid x, \psi) = P_{\mathrm{LLM}}(\cdot \mid x, \psi)$ and $P(\cdot \mid x) = P_{\mathrm{LLM}}(\cdot \mid x)$, and let $\mu_\xi^S(z)$ be the same quantity for a prompt in which the descriptions are randomly permuted across the rollouts of the landscape, with a fresh permutation for every query. The *feedback effect* and its pairing and text parts are

$$\Delta_\xi(z) = \mu_\xi^Q(z) - \mu_\xi^P(z) = \underbrace{\mu_\xi^Q(z) - \mu_\xi^S(z)}_{\Delta_\xi^{\mathrm{pair}}(z)} + \underbrace{\mu_\xi^S(z) - \mu_\xi^P(z)}_{\Delta_\xi^{\mathrm{text}}(z)}. \tag{4}$$

At a stored context of a BMPS run, $\Delta_\xi(z)$ is estimated without bias by querying the LLM n times with each prompt and evaluating each proposal with a fresh rollout, and by Hoeffding's inequality the estimate exceeds $\Delta_\xi(z)$ by more than $\sqrt{\log(1/\delta)/n}$ with probability at most δ. Proposition D.1 in Appendix D.1 relates the effect to the decision that the descriptions inform: $\mathbb{E}[\Delta_\xi(Z_t) \mid X_t = x] = \mathcal{I}_t(x) - \mathcal{R}_{1,t}(x) + \mathcal{R}_{0,t}(x)$. Here the *decision value* $\mathcal{I}_t \geq 0$ is what an ideal decision maker gains by also seeing the descriptions, and it is positive only if the descriptions can overturn the decision that the numerical evidence supports, while $\mathcal{R}_{1,t}$ and $\mathcal{R}_{0,t}$ are the proposal regrets of the LLM with and without them. A negative effect therefore reveals a loss in how the LLM uses the descriptions rather than a lack of information in them. Since Assumption 4.2 allows any distribution of ξ, taking it to be the distribution of the experimental tasks and seeds makes the average of such estimates over contexts of independent runs an unbiased estimate of the average gain in expected return of BMPS's proposals over numerical-only proposals made at the same contexts. The shuffled prompt has the length and the descriptions of the BMPS prompt but not their pairing with parameters and returns. Shuffled descriptions are a garbling of the paired ones [3], so their decision value lies between 0 and $\mathcal{I}_t$: $\Delta_\xi^{\mathrm{pair}}$ isolates what the anchor–neighbor pairing contributes, and $\Delta_\xi^{\mathrm{text}}$ what adding behavioral text contributes, including any harm from a longer prompt.

4.2 Sublinear Bayesian Regret

The feedback effect concerns a single iteration and does not by itself imply sublinear growth of BR_T. No such guarantee can hold without a condition on the LLM, since an LLM that always proposes the same suboptimal policy has linear regret. We first isolate the quantity that determines the growth rate. Let $H(\theta^* \mid A)$ be the entropy of the conditional distribution of θ^* given an event A, and let $H(\theta^* \mid A, Y)$ be its average over the values of a random variable Y (Appendix C). For a record $z_{1:t}$ of positive probability, let $A = \{Z_{1:t} = z_{1:t}\}$, and define the expected regret of iteration t, the information that it reveals about the best policy, and the information ratio [25],

$$\begin{aligned} \varrho_t(z_{1:t}) &= \mathbb{E}\big[j_\xi(\theta^*) - j_\xi(\theta_t) \,\big|\, A\big], \\ \iota_t(z_{1:t}) &= H(\theta^* \mid A) - H(\theta^* \mid A, Z_{t+1}), \\ \Gamma_t(z_{1:t}) &= \varrho_t(z_{1:t})^2/\iota_t(z_{1:t}), \end{aligned} \tag{5}$$

with $0/0 = 0$ and $a/0 = +\infty$ for $a > 0$. The quantity $\iota_t \geq 0$ is the conditional mutual information between θ^* and the next record, which contains the evaluation of θ_t and the next neighborhood with its descriptions.

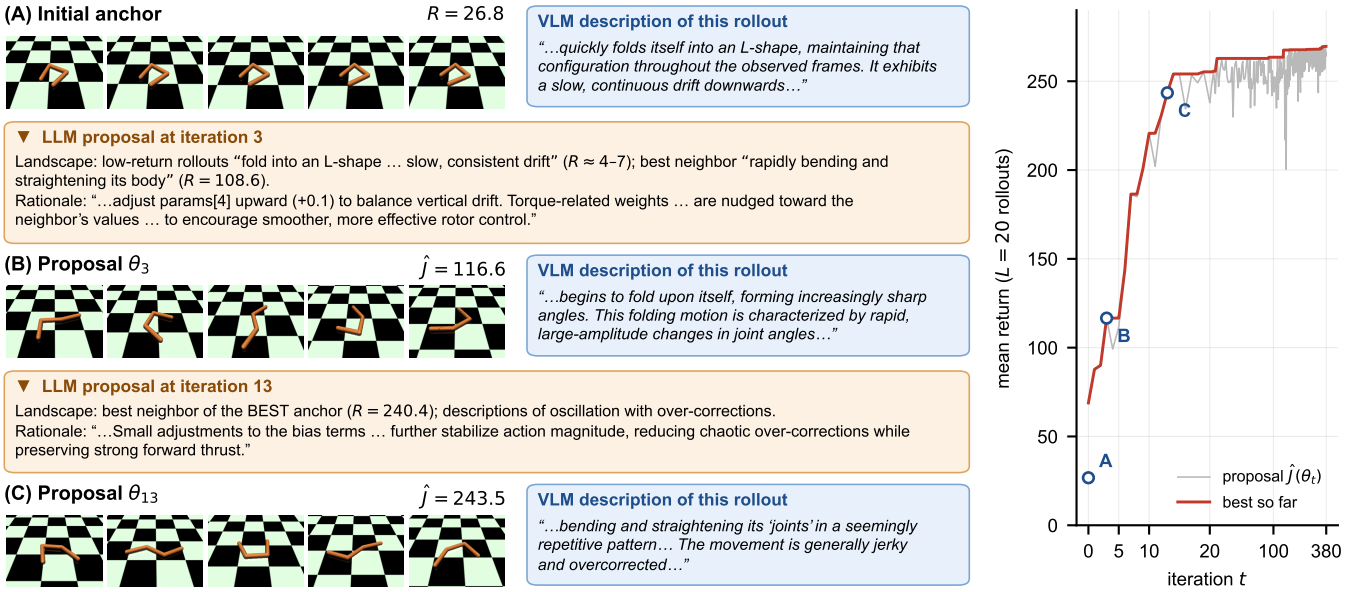


Figure 2: Behavior-guided search on *Swimmer* (one BMPS run). (A)–(C): rollouts of the initial anchor and of the proposals at iterations 3 and 13, each with a verbatim excerpt of the VLM description of that rollout. Orange boxes: excerpts of the behavioral landscape that the LLM received and of the rationale it returned with its proposal. R is the return of the described rollout, $\hat{J}$ the mean over $L = 20$ evaluation rollouts. Frames are re-rendered from the logged parameters at steps $200, 400, \ldots, 1000$. Right: evaluated return of each proposal and the best return so far (symmetric-log horizontal axis).

The analysis of [25] applies to any such loop. The information gains telescope, $\sum_t \mathbb{E}[\iota_t(Z_{1:t})] \le H(\theta^* \mid Z_1) \le \log|C|$, and the Cauchy–Schwarz inequality gives $\mathrm{BR}_T \le \sqrt{\overline{\Gamma}_T\, T \log|C|}$, where $\overline{\Gamma}_T$ is the average of $\mathbb{E}[\Gamma_t(Z_{1:t})]$ over $t \le T$. Search loops over C therefore differ only through the squared regret that they incur per unit of information about θ^*. Behavioral feedback acts on both terms of this ratio. For a fixed proposal at iteration t, recording Ψ_{t+1} in addition to X_{t+1} can only increase ι_t, because conditioning reduces entropy (Lemma C.2). The descriptions also change the proposal: if it were drawn from the numerical prompt, the regret of iteration t would change by $\mathbb{E}[j_\xi(\theta_t) - j_\xi(\theta_t^P) \mid Z_{1:t}]$, whose expectation is $\mathbb{E}[\Delta_\xi(Z_t)]$. Feedback lowers the ratio when both changes are favorable, and neither the theory nor a single measurement establishes both. The argument is also not specific to vision, since any additional record increases ι_t. A rate requires a condition on the LLM.

Let $\pi_t(\theta) = \Pr(\theta^* = \theta \mid A)$ be the posterior of the best policy given the record; Thompson sampling [33] would propose $\theta_t \sim \pi_t$. For a candidate θ with $\Pr(A, \theta_t = \theta) > 0$, let $A_\theta = A \cap \{\theta_t = \theta\}$, let $\iota_t(\theta; z_{1:t}) = H(\theta^* \mid A_\theta) - H(\theta^* \mid A_\theta, Z_{t+1})$ be the information that the next record reveals when θ is proposed, and let $\varrho_t^\pi(z_{1:t}) = \mathbb{E}[j_\xi(\theta^*) \mid A] - \sum_{\theta \in C} \pi_t(\theta)\, \mathbb{E}[j_\xi(\theta) \mid A]$ be the expected regret of Thompson sampling.

Assumption 4.3 (Approximate posterior sampling). *There are $c \ge 1$ and $\varepsilon_t(z_{1:t}) \in [0,1]$ such that, for every t and every record $z_{1:t}$ of positive probability, the proposal distribution $Q_t = P_{\mathrm{LLM}}(\cdot \mid z_t)$ satisfies $Q_t(\theta) \ge \pi_t(\theta)/c$ for every $\theta \in C$, and $\mathrm{TV}(Q_t, \pi_t) \le \varepsilon_t(z_{1:t})$.*

The first condition requires that the LLM not rule out policies that may be optimal given the record, and the second that it approaches posterior sampling as $\varepsilon_t \to 0$. Neither condition can be verified from samples alone, and Assumption 4.3 is used only in the following theorem.

Theorem 4.4 (Posterior sampling). *Let Assumptions 4.1–4.3 hold, and suppose that, for every record $z_{1:t}$ of positive probability,*

$$\varrho_t^\pi(z_{1:t})^2 \le \Gamma^\circ \sum_{\theta \in C} \pi_t(\theta)\, \iota_t(\theta; z_{1:t}). \tag{6}$$

Then

$$\mathrm{BR}_T \le \sqrt{c\,\Gamma^\circ\, T \log|C|} + \sum_{t=1}^{T} \mathbb{E}\big[\varepsilon_t(Z_{1:t})\big]. \tag{7}$$

If C is partitioned into M groups such that, whenever θ_t lies in a group G, the record Z_{t+1} contains, for each $\theta' \in G$, a statistic in $[0,1]$ whose conditional mean given ξ and the past is $j_\xi(\theta')$, then Eq. (6) holds with $\Gamma^\circ = M/2$.

The bound is sublinear whenever $\sum_t \mathbb{E}[\varepsilon_t] = o(T)$. The constant $\Gamma^\circ = M/2$ interpolates between bandit feedback, where only the return of the proposal is revealed ($M = |C|$, which recovers the ratio $|C|/2$ of Thompson sampling [25]), and full information ($M = 1$). Under the idealized group-feedback condition, observing unbiased statistics for a whole group of policies yields $M/2$ with M the number of groups, and this is how the behavioral landscape would enter the rate. For the grids of Section 3.1, $|C| = 121^d$, so the bound is informative only when each iteration reveals returns over large regions; the group condition idealizes anchor-centered sampling and is not a property that BMPS guarantees.

4.3 Quality of the Returned Policy

In policy search, intermediate proposals are evaluations rather than deployments, and what matters is the returned policy. BMPS returns the evaluated policy $\widehat{\theta}_T$ with the best recorded return, and for its simple regret a much weaker condition than Assumption 4.3 suffices. Fix the environment, let θ^* maximize j over Θ, let $\mathcal{S}_\varepsilon = \{\theta : j(\theta) \geq j(\theta^*) - \varepsilon\}$ be the set of ε-optimal policies, and let $q_t = Q_t(\mathcal{S}_\varepsilon)$ be the probability that the proposal of iteration t is ε-optimal.

Theorem 4.5 (Simple regret of the returned policy). *Let Assumption 4.1 hold, and suppose that each of the K evaluated policies, among them $\theta_1, \ldots, \theta_T$, has a recorded normalized return computed from L fresh rollouts. For every $s > 0$ and $\delta \in (0, 1)$, with probability at least $1 - e^{-s} - \delta$, either $\sum_{t=1}^{T} q_t < s$ or*

$$j(\theta^*) - j(\widehat{\theta}_T) \leq \varepsilon + 2\sqrt{\frac{\log(2K/\delta)}{2L}}. \tag{8}$$

Moreover, $q_t \geq 1 - r_t/\varepsilon$ for $\varepsilon > 0$, where $r_t = j(\theta^*) - \mathbb{E}_{Q_t}[j]$ is the expected regret of the proposal of iteration t.

The theorem tolerates arbitrarily many poor proposals. If the proposals keep a nonvanishing probability of being ε-optimal, say $q_t \geq q$, then $\sum_t q_t \geq qT$, and the returned policy is ε-optimal up to the estimation error with probability at least $1 - e^{-qT} - \delta$. Because K grows with T, the number L of evaluation rollouts must grow logarithmically to keep the estimation term fixed. Behavioral feedback enters through q_t: by the last statement, the lower bound on q_t increases as the expected proposal regret decreases. In the fixed environment of the theorem, this link to the measurement is direct and needs no distribution over environments. The expected proposal regrets with and without descriptions satisfy $r_t = r_t^P - \Delta_\xi(Z_t)$, where $r_t^P = j(\theta^*) - \mathbb{E}_{P_t}[j]$ and Δ_ξ is the feedback effect of Eq. (4), so that

$$q_t \geq 1 - \big(r_t^P - \Delta_\xi(Z_t)\big)/\varepsilon. \tag{9}$$

5 Experiments

5.1 Experimental Setting

Benchmark suite. We evaluate BMPS on a diverse suite of Gymnasium environments spanning both continuous and discrete control tasks. The benchmark is designed to provide broad coverage across different control settings, enabling a comprehensive evaluation of reasoning-driven policy search.

Compared baselines. We compare BMPS against a diverse set of baselines spanning policy gradient, evolutionary strategies, and prior LLM-based approaches. Our work is directly inspired by ProPS and ProPS+ [38], which pioneered the use of LLMs as black-box optimizers for RL policy search. ProPS represents the foundational variant, wherein the LLM is prompted to refine policies using only interaction history (state-action-reward sequences). ProPS+ extends this by incorporating environment descriptions as structured prior knowledge. While these methods demonstrate the potential of language-guided optimization, they rely exclusively on scalar reward signals for feedback. BMPS builds upon this foundation by augmenting the optimization loop with multimodal behavioral diagnostics from vision-language models, enabling diagnosis-aware policy refinement beyond reward-only prompting.

For policy-gradient and value-based reinforcement learning methods, we include TRPO [27], PPO [28], A2C [20], and DQN [19].. For evolutionary strategies, we evaluate OpenAI-ES [26], which estimates gradients via population perturbations; CMA-ES [8], which adapts a full covariance matrix to model the search distribution; the classical (μ, λ)-ES, which selects the top μ individuals from λ offspring; and ARS [17], a simple random-search method that directly perturbs policy parameters. All baseline methods are trained for a fixed budget of 8×10^6 environment interaction steps to ensure fair comparison across approaches. Each run stops after 400 iterations, 8×10^6 environment steps including neighborhood rollouts, or when the prompt exceeds the LLM context window, whichever comes first.

LLM and VLM Setting. For all LLM-based methods, including BMPS, we use GPT-OSS-120B [23] as the LLM and Gemma 3 27B-IT [32] as the VLM. The temperature for both the LLM and VLM is set to 1.

5.2 Performance on Continuous Control Tasks

Table 1 reports results on seven continuous-control benchmarks. BMPS achieves the highest mean return on four environments (Mountain Car Continuous, Inverted Pendulum, Swimmer, Walker). On Reacher, its mean return is close to PPO's (−7.36 versus −7.32). However, on Hopper, BMPS reaches 703.97 ± 323.75, compared with CMA-ES at 1677.37 ± 612.72. On Inverted Double Pendulum, BMPS has a lower mean than TRPO (982.31 versus 3609.37) amid large run-to-run variability.

Variability also depends on the task and comparator. BMPS records 1000.00 ± 0.00 on Inverted Pendulum and a lower standard deviation than the evolutionary baselines on Swimmer (57.38, compared with 157.72 for CMA-ES). In contrast, its standard deviation on Inverted Double Pendulum is 2482.35, exceeding its mean of 982.31, and variability remains substantial on Hopper (323.75). These results do not establish a general reduction in variance or sensitivity to hyperparameters and initialization.

5.3 Performance on Discrete Control Tasks

Table 2 summarizes performance on six discrete-action benchmarks. BMPS achieves the highest mean return on Frozen Lake, Cliff Walking, Mountain Car, and Pong, and ties for best on Maze. On Navigation, BMPS has a higher mean than the other LLM-based and evolutionary methods but a lower mean than A2C, PPO, and TRPO. Its return is 2279.48 ± 1172.74, compared with TRPO's 4223.51 ± 19.70, indicating both a lower mean and substantial run-to-run variability.

Frozen Lake and Mountain Car illustrate gains on tasks with sparse or initially uninformative returns. On Frozen Lake, BMPS achieves 0.975 ± 0.025, compared with 0.15 for A2C and 0.16 for PPO. On Mountain Car, BMPS achieves -112.37 ± 2.58, compared with ProPS+ at -117.47 ± 23.35, with a higher mean and a lower standard deviation. These observations are consistent with behavioral feedback helping search on some tasks, but the return tables alone do not establish that it prevents local optima or premature convergence.

Table 1: Performance on continuous control environments. Policy-gradient baselines follow [38] (exact same setting).

Method	Mtn. Car Cont.	Inv. Pendulum	Inv. Dbl. Pendulum	Reacher	Swimmer	Hopper	Walker
Policy Gradient Methods							
A2C	74.10 ± 7.44	155.15 ± 53.79	102.90 ± 32.04	-53.48 ± 21.42	39.40 ± 6.54	123.49 ± 86.46	355.84 ± 154.02
PPO	78.16 ± 5.32	218.65 ± 129.31	108.60 ± 4.12	**-7.32 ± 0.38**	37.31 ± 7.19	351.75 ± 157.71	469.78 ± 159.17
TRPO	1.18 ± 3.54	571.31 ± 358.88	**3609.37 ± 4000.04**	-8.93 ± 1.39	52.96 ± 18.86	716.90 ± 385.20	519.38 ± 73.15
LLM-based Methods							
ProPS	99.09 ± 0.18	819.17 ± 362.37	125.89 ± 32.67	-7.95 ± 0.58	151.64 ± 56.03	407.86 ± 301.35	283.88 ± 65.17
ProPS+	97.92 ± 1.90	882.19 ± 238.13	144.52 ± 69.90	-7.61 ± 0.35	210.65 ± 58.89	421.35 ± 294.93	318.46 ± 45.07
Evolutionary Computing Methods							
OpenAI-ES	9.80 ± 29.72	364.94 ± 417.36	91.78 ± 4.54	-244.79 ± 335.38	114.22 ± 95.34	975.21 ± 174.19	-6.52 ± 4.67
CMA-ES	30.90 ± 44.70	604.07 ± 484.97	1052.25 ± 2769.83	-9.60 ± 1.16	205.14 ± 157.72	**1677.37 ± 612.72**	145.67 ± 297.01
(μ, λ)**-ES**	37.95 ± 47.08	477.49 ± 441.24	113.10 ± 99.16	-21.71 ± 31.56	167.73 ± 126.58	1224.61 ± 404.43	-13.30 ± 8.09
ARS	9.59 ± 29.81	216.80 ± 393.03	151.46 ± 229.83	-39.12 ± 49.89	51.70 ± 88.47	283.13 ± 339.84	-41.94 ± 22.69
Ours							
BMPS	**99.13 ± 0.10**	**1000.00 ± 0.00**	982.31 ± 2482.35	-7.36 ± 0.66	**289.47 ± 57.38**	703.97 ± 323.75	**552.02 ± 175.39**

Table 2: Performance on discrete control environments. Policy-gradient baselines follow [38] (exact same setting).

Method	Frozen Lake	Cliff Walking	Maze	Mtn. Car	Navigation	Pong
Policy Gradient Methods						
A2C	0.15 ± 0.03	-172.30 ± 14.49	**0.97 ± 0.00**	-200.00 ± 0.00	3670.87 ± 124.29	0.57 ± 0.14
DQN	0.13 ± 0.03	-320.93 ± 29.60	0.86 ± 0.22	-194.36 ± 1.47	-127.59 ± 7.13	0.56 ± 0.02
PPO	0.16 ± 0.02	-94.35 ± 3.96	0.96 ± 0.00	-200.00 ± 0.00	4127.43 ± 24.29	2.29 ± 0.91
TRPO	0.22 ± 0.05	-66.60 ± 13.61	**0.97 ± 0.00**	-200.00 ± 0.00	**4223.51 ± 19.70**	1.36 ± 1.05
LLM-based Methods						
ProPS	0.29 ± 0.09	-100.00 ± 0.00	**0.97 ± 0.00**	-190.58 ± 28.28	2003.38 ± 1094.46	1.89 ± 0.98
ProPS+	0.925 ± 0.026	-69.73 ± 8.51	**0.97 ± 0.00**	-117.47 ± 23.35	1863.72 ± 1240.19	2.65 ± 0.43
Evolutionary Computing Methods						
OpenAI-ES	0.22 ± 0.15	-540.10 ± 922.00	-1.11 ± 0.00	-200.00 ± 0.00	136.75 ± 768.11	1.11 ± 0.62
CMA-ES	0.16 ± 0.19	-235.30 ± 201.40	-1.11 ± 0.00	-200.00 ± 0.00	575.25 ± 1273.55	1.07 ± 0.97
(μ, λ)**-ES**	0.24 ± 0.07	-108.30 ± 11.20	-1.11 ± 0.00	-200.00 ± 0.00	669.63 ± 1216.00	0.77 ± 0.86
ARS	0.16 ± 0.11	-271.80 ± 161.50	-0.90 ± 0.62	-200.00 ± 0.00	-113.33 ± 94.17	0.22 ± 0.07
Ours						
BMPS	**0.975 ± 0.025**	**-64.35 ± 5.98**	**0.97 ± 0.00**	**-112.37 ± 2.58**	2279.48 ± 1172.74	**2.96 ± 0.11**

5.4 Ablation Studies

To better understand how different components of BMPS contribute to search performance, we conduct ablation studies on *Swimmer* and discrete *MountainCar*, which serve as representative benchmarks for continuous and discrete control, respectively.

Effect of Environment Prior Knowledge. We first examine the contribution of textual prior knowledge about the environment by comparing the full method with a variant whose LLM prompt omits the environment description. In Table 3, full BMPS has a higher mean return and lower standard deviation on both tested tasks.

Table 3: Ablation of environment prior knowledge.

Method	*Swimmer*	*MountainCar*
w/o environment prior	206.325 ± 103.107	-142.055 ± 40.751
BMPS	**309.36 ± 51.83**	**-112.01 ± 2.85**

These results suggest a benefit from environment context in these settings.

Table 4: Ablation of contextual and anchor-policy components.

Method	*Swimmer*	*MountainCar*
w/o optimization history	270.95 ± 21.63	-133.95 ± 28.20
w/o neighbor sampling	202.07 ± 59.38	-135.26 ± 33.75
w/o current anchor	253.14 ± 70.36	-125.75 ± 25.72
w/o best anchor	258.55 ± 84.58	-141.09 ± 34.42
w/o worst anchor	268.02 ± 90.37	-117.7 ± 20.69
BMPS	**309.36 ± 51.83**	**-112.01 ± 2.85**

Table 5: Ablation study on the number of neighbors in BMPS.

Method	*Swimmer*	*MountainCar*
0 neighbor	202.07 ± 59.38	-135.26 ± 33.75
3 neighbors	215.35 ± 90.08	-140.93 ± 36.88
7 neighbors	275.64 ± 87.53	-119.71 ± 20.23
BMPS (5 neighbors)	**309.36 ± 51.83**	**-112.01 ± 2.85**

Effect of Visual Feedback and Selection of Anchor Policies. We next study the contribution of the multimodal diagnosis module by removing individual components from the full method. As shown in Table 4, we evaluate variants without optimization history, without neighbor sampling, and without each anchor policy type (current anchor, best anchor, and worst anchor), in order to identify which sources of contextual and behavioral information are most important for stable policy improvement. Here, optimization history refers to the summary of previous policy proposals and their observed returns provided to the LLM prompt. Removing this component tests whether iterative search in BMPS benefits from remembering past trials rather than relying only on the current round of multimodal feedback.

Effect of the Number of Neighbors. Finally, we vary the number of neighboring policies sampled around each anchor. Among the tested settings in Table 5, five neighbors yields the highest mean return and lowest standard deviation on both environments, and we use this setting in the main experiments. Performance does not improve monotonically with the number of neighbors: three neighbors has a lower mean than zero, while seven has a lower mean than five. These comparisons support the choice of five neighbors for the two tested tasks, but do not establish a general optimum or identify whether redundant or conflicting feedback explains the differences.

5.5 Case Study: How Behavioral Feedback Guides the Search

Figure 2 traces how the behavioral landscape enters the proposals of one BMPS run on *Swimmer*. The initial anchor folds into an L-shape and holds it while drifting ($R = 26.8$), and the VLM describes exactly this posture. At iteration 3, the landscape contrasts low-return rollouts that "fold into an L-shape" and drift with a neighbor that keeps "bending and straightening its body" ($R = 108.6$), and the LLM moves toward that neighbor while citing vertical drift and rotor control in its rationale. The resulting policy θ_3 folds and unfolds with large amplitude and reaches $\hat{J} = 116.6$. At iteration 13, the descriptions report oscillation with over-corrections, and the LLM adjusts the bias terms to "stabilize action magnitude, reducing chaotic over-corrections while preserving strong forward thrust"; θ_{13} settles into a repetitive bending pattern and reaches $\hat{J} = 243.5$, close to the best return of the run. The rationales also rely on the returns of the neighbors, so the trace illustrates how behavioral and numerical evidence are combined in a proposal rather than isolating the effect of either.

6 Conclusion

We introduced BMPS, a multimodal policy search framework that enriches LLM-based optimization with structured visual feedback from a VLM. By combining reward histories with trajectory-level behavioral diagnostics, BMPS provides behavioral evidence for policy proposals. In the 13-task evaluation, it achieves the best average rank and higher mean returns than ProPS and ProPS+ on 12 tasks, tying on Maze. Its advantages over policy-gradient and evolutionary methods depend on the task: it trails leading baselines on Hopper, Navigation, and Inverted Double Pendulum, and substantial run-to-run variability remains on some tasks.

A key limitation is token efficiency: augmenting prompts with optimization history, neighbor comparisons, and VLM feedback is substantially more costly than reward-only baselines, and scaling to longer trajectories or richer observations may require aggressive prompt compression. Looking ahead, an important direction is formalizing the optimization loop as a sequential decision-making problem, which would provide a principled foundation for managing the exploration–exploitation trade-off and reducing context overhead over time.

References

[1] Heni Ben Amor, Laura Graesser, Atil Iscen, David D'Ambrosio, Saminda Abeyruwan, Alex Bewley, Yifan Zhou, Kamalesh Kalirathinam, Swaroop Mishra, and Pannag Sanketi. 2025. Sas-Prompt: Large Language Models as Numerical Optimizers for Robot Self-Improvement. arXiv:2504.20459 [cs.RO] https://arxiv.org/abs/2504.20459

[2] Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. 2000. A Neural Probabilistic Language Model. In *Advances in Neural Information Processing Systems*, T. Leen, T. Dietterich, and V. Tresp (Eds.), Vol. 13. MIT Press. https://proceedings.neurips.cc/paper_files/paper/2000/file/728f206c2a01bf572b5940d7d9a8fa4c-Paper.pdf

[3] David Blackwell. 1953. Equivalent Comparisons of Experiments. *The Annals of Mathematical Statistics* 24, 2 (1953), 265–272.

[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs.CL] https://arxiv.org/abs/2005.14165

[5] Dingyang Chen, Qi Zhang, and Yinglun Zhu. 2025. Efficient Sequential Decision Making with Large Language Models. arXiv:2406.12125 [cs.LG] https://arxiv.org/abs/2406.12125

[6] Prasoon Goyal, Scott Niekum, and Raymond J. Mooney. 2021. PIXL2R: Guiding Reinforcement Learning using Natural Language by Mapping Pixels to Rewards. In *Proceedings of the Conference on Robot Learning*.

[7] Yanqiao Han, Menglong Yang, Yang Ren, and Weizheng Li. 2024. Large Language Model Guided Reinforcement Learning Based Six-Degree-of-Freedom Flight Control. *IEEE Access* 12 (2024), 89479–89492. doi:10.1109/ACCESS.2024.3411015

[8] Nikolaus Hansen. 2023. The CMA Evolution Strategy: A Tutorial. arXiv:1604.00772 [cs.LG] https://arxiv.org/abs/1604.00772

[9] Sinan Ibrahim, Mostafa Mostafa, Ali Jnadi, Hadi Salloum, and Pavel Osinenko. 2024. Comprehensive Overview of Reward Engineering and Shaping in Advancing Reinforcement Learning Applications. *IEEE Access* 12 (2024), 175473–175500.

[10] Gregory Kuhlmann, Peter Stone, Raymond J. Mooney, and Jude W. Shavlik. 2004. Guiding a Reinforcement Learner with Natural Language Advice: Initial Results

in RoboCup Soccer. In *AAAI Workshop on Supervisory Control of Learning and Adaptive Systems*.

[11] Minae Kwon, Sang Michael Xie, Kalesha Bullard, and Dorsa Sadigh. 2023. Reward Design with Language Models. In *International Conference on Learning Representations*.

[12] Michael Laskin, Luyu Wang, Junhyuk Oh, Emilio Parisotto, Stephen Spencer, Richie Steigerwald, DJ Strouse, Steven Hansen, Angelos Filos, Ethan Brooks, Maxime Gazeau, Himanshu Sahni, Satinder Singh, and Volodymyr Mnih. 2023. In-Context Reinforcement Learning with Algorithm Distillation. In *International Conference on Learning Representations*.

[13] Jacky Liang, Fei Xia, Wenhao Yu, Andy Zeng, Montserrat Gonzalez Arenas, Maria Attarian, Maria Bauza, Matthew Bennice, Alex Bewley, Adil Dostmohamed, Chuyuan Kelly Fu, Nimrod Gileadi, Marissa Giustina, Keerthana Gopalakrishnan, Leonard Hasenclever, Jan Humplik, Jasmine Hsu, Nikhil Joshi, Ben Jyenis, Chase Kew, Sean Kirmani, Tsang-Wei Edward Lee, Kuang-Huei Lee, Assaf Hurwitz Michaely, Joss Moore, Ken Oslund, Dushyant Rao, Allen Ren, Baruch Tabanpour, Quan Vuong, Ayzaan Wahid, Ted Xiao, Ying Xu, Vincent Zhuang, Peng Xu, Erik Frey, Ken Caluwaerts, Tingnan Zhang, Brian Ichter, Jonathan Tompson, Leila Takayama, Vincent Vanhoucke, Izhak Shafran, Maja Mataric, Dorsa Sadigh, Nicolas Heess, Kanishka Rao, Nik Stewart, Jie Tan, and Carolina Parada. 2024. Learning to Learn Faster from Human Feedback with Language Model Predictive Control. In *Robotics: Science and Systems*.

[14] Shaoteng Liu, Haoqi Yuan, Minda Hu, Yanwei Li, Yukang Chen, Shu Liu, Zongqing Lu, and Jiaya Jia. 2024. RL-GPT: Integrating Reinforcement Learning and Code-as-Policy. In *Advances in Neural Information Processing Systems*.

[15] Jelena Luketina, Nantas Nardelli, Gregory Farquhar, Jakob Foerster, Jacob Andreas, Edward Grefenstette, Shimon Whiteson, and Tim Rockt"aschel. 2019. A Survey of Reinforcement Learning Informed by Natural Language. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*.

[16] Yecheng Jason Ma, William Liang, Hungju Wang, Sam Wang, Yuke Zhu, Linxi Fan, Osbert Bastani, and Dinesh Jayaraman. 2024. DrEureka: Language Model Guided Sim-to-Real Transfer. In *Robotics: Science and Systems*.

[17] Horia Mania, Aurelia Guy, and Benjamin Recht. 2018. Simple random search of static linear policies is competitive for reinforcement learning. In *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.), Vol. 31. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2018/file/7634ea65a4e6d9041cfd3f7de18e334a-Paper.pdf

[18] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing Atari with Deep Reinforcement Learning. arXiv:1312.5602 [cs.LG] https://arxiv.org/abs/1312.5602

[19] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533. doi:10.1038/nature14236

[20] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous Methods for Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. 1928–1937.

[21] Giovanni Monea, Antoine Bosselut, Kianté Brantley, and Yoav Artzi. 2025. LLMs are In-Context Bandit Reinforcement Learners. arXiv:2410.05362 [cs.LG] https://arxiv.org/abs/2410.05362

[22] Karthik Narasimhan, Regina Barzilay, and Tommi Jaakkola. 2018. Grounding Language for Transfer in Deep Reinforcement Learning. *Journal of Artificial Intelligence Research* 63 (2018), 849–874.

[23] OpenAI, :, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, Che Chang, Kai Chen, Mark Chen, Enoch Cheung, Aidan Clark, Dan Cook, Marat Dukhan, Casey Dvorak, Kevin Fives, Vlad Fomenko, Timur Garipov, Kristian Georgiev, Mia Glaese, Tarun Gogineni, Adam Goucher, Lukas Gross, Katia Gil Guzman, John Hallman, Jackie Hehir, Johannes Heidecke, Alec Helyar, Haitang Hu, Romain Huet, Jacob Huh, Saachi Jain, Zach Johnson, Chris Koch, Irina Kofman, Dominik Kundel, Jason Kwon, Volodymyr Kyrylov, Elaine Ya Le, Guillaume Leclerc, James Park Lennon, Scott Lessans, Mario Lezcano-Casado, Yuanzhi Li, Zhuohan Li, Ji Lin, Jordan Liss, Lily, Liu, Jiancheng Liu, Kevin Lu, Chris Lu, Zoran Martinovic, Lindsay McCallum, Josh McGrath, Scott McKinney, Aidan McLaughlin, Song Mei, Steve Mostovoy, Tong Mu, Gideon Myles, Alexander Neitz, Alex Nichol, Jakub Pachocki, Alex Paino, Dana Palmie, Ashley Pantuliano, Giambattista Parascandolo, Jongsoo Park, Leher Pathak, Carolina Paz, Ludovic Peran, Dmitry Pimenov, Michelle Pokrass, Elizabeth Proehl, Huida Qiu, Gaby Raila, Filippo Raso, Hongyu Ren, Kimmy Richardson, David Robinson, Bob Rotsted, Hadi Salman, Suvansh Sanjeev, Max Schwarzer, D. Sculley, Harshit Sikchi, Kendal Simon, Karan Singhal, Yang Song, Dane Stuckey, Zhiqing Sun, Philippe Tillet, Sam Toizer, Foivos Tsimpourlas, Nikhil Vyas, Eric Wallace, Xin Wang, Miles Wang, Olivia Watkins, Kevin Weil, Amy Wendling, Kevin Whinnery, Cedric Whitney, Hannah Wong, Lin Yang, Yu Yang, Michihiro Yasunaga, Kristen Ying, Wojciech Zaremba, Wenting Zhan, Cyril Zhang, Brian Zhang, Eddie Zhang, and Shengjia Zhao. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925 [cs.CL] https://arxiv.org/abs/2508.10925

[24] OpenAI, :, Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson, Jakub Pachocki, Michael Petrov, Henrique P. d. O. Pinto, Jonathan Raiman, Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang, Filip Wolski, and Susan Zhang. 2019. Dota 2 with Large Scale Deep Reinforcement Learning. arXiv:1912.06680 [cs.LG] https://arxiv.org/abs/1912.06680

[25] Daniel Russo and Benjamin Van Roy. 2016. An Information-Theoretic Analysis of Thompson Sampling. *Journal of Machine Learning Research* 17, 68 (2016), 1–30.

[26] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. 2017. Evolution Strategies as a Scalable Alternative to Reinforcement Learning. arXiv:1703.03864 [stat.ML] https://arxiv.org/abs/1703.03864

[27] John Schulman, Sergey Levine, Pieter Abbeel, Michael I. Jordan, and Philipp Moritz. 2015. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. 1889–1897.

[28] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347 [cs.LG] https://arxiv.org/abs/1707.06347

[29] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. 2016. Mastering the game of Go with deep neural networks and tree search. *Nature* 529, 7587 (2016), 484–489. doi:10.1038/nature16961

[30] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. 2017. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. arXiv:1712.01815 [cs.AI] https://arxiv.org/abs/1712.01815

[31] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement learning: An introduction*. MIT Press.

[32] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesht Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alex Feng, Alexander Kolesnikov, Alexei Bendebury, Alvin Abdagic, Amit Vadi, András György, André Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Bo Wu, Bobak Shahriari, Bryce Petrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, CJ Carey, Cormac Brick, Daniel Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Eric Noland, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirkhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Plucińska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szpektor, Ivan Nardini, Jean Pouget-Abadie, Jetha Chan, Joe Stanton, John Wieting, Jonathan Lai, Jordi Orbay, Joseph Fernandez, Josh Newlan, Ju yeong Ji, Jyotinder Singh, Kat Black, Kathy Yu, Kevin Hui, Kiran Vodrahalli, Klaus Greff, Linhai Qiu, Marcella Valentine, Marina Coelho, Marvin Ritter, Matt Hoffman, Matthew Watson, Mayank Chaturvedi, Michael Moynihan, Min Ma, Nabila Babar, Natasha Noy, Nathan Byrd, Nick Roy, Nikola Momchev, Nilay Chauhan, Noveen Sachdeva, Oskar Bunyan, Pankil Botarda, Paul Caron, Paul Kishan Rubenstein, Phil Culliton, Philipp Schmid, Pier Giuseppe Sessa, Pingmei Xu, Piotr Stanczyk, Pouya Tafti, Rakesh Shivanna, Renjie Wu, Renke Pan, Reza Rokni, Rob Willoughby, Rohith Vallu, Ryan Mullins, Sammy Jerome, Sara Smoot, Sertan Girgin, Shariq Iqbal, Shashir Reddy, Shruti Sheth, Siim Põder, Sijal Bhatnagar, Sindhu Raghuram Panyam, Sivan Eiger, Susan Zhang, Tianqi Liu, Trevor Yacovone, Tyler Liechty, Uday Kalra, Utku Evci, Vedant Misra, Vincent Roseberry, Vlad Feinberg, Vlad Kolesnikov, Woohyun Han, Woosuk Kwon, Xi Chen, Yinlam Chow, Yuvein Zhu, Zichuan Wei, Zoltan Egyed, Victor Cotruta, Minh Giang, Phoebe Kirk, Anand Rao, Kat Black, Nabila Babar, Jessica Lo, Erica Moreira, Luiz Gustavo Martins, Omar Sanseviero, Lucas Gonzalez, Zach Gleicher, Tris Warkentin, Vahab Mirrokni, Evan Senter, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, Yossi Matias, D. Sculley, Slav Petrov, Noah Fiedel, Noam Shazeer, Oriol Vinyals, Jeff

Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Jean-Baptiste Alayrac, Rohan Anil, Dmitry, Lepikhin, Sebastian Borgeaud, Olivier Bachem, Armand Joulin, Alek Andreev, Cassidy Hardin, Robert Dadashi, and Léonard Hussenot. 2025. Gemma 3 Technical Report. arXiv:2503.19786 [cs.CL] https://arxiv.org/abs/2503.19786

[33] William R. Thompson. 1933. On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples. *Biometrika* 25, 3/4 (1933), 285–294.

[34] Xiefeng Wu. 2024. Enhancing Q-Learning with Large Language Model Heuristics. arXiv:2405.03341 [cs.LG] https://arxiv.org/abs/2405.03341

[35] Xue Yan, Yan Song, Xidong Feng, Mengyue Yang, Haifeng Zhang, Haitham Bou Ammar, and Jun Wang. 2024. Efficient Reinforcement Learning with Large Language Model Priors. arXiv:2410.07927 [cs.LG] https://arxiv.org/abs/2410.07927

[36] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large Language Models as Optimizers. arXiv:2309.03409 [cs.LG] https://arxiv.org/abs/2309.03409

[37] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montse Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, Brian Ichter, Ted Xiao, Peng Xu, Andy Zeng, Tingnan Zhang, Nicolas Heess, Dorsa Sadigh, Jie Tan, Yuval Tassa, and Fei Xia. 2023. Language to Rewards for Robotic Skill Synthesis. arXiv:2306.08647 [cs.RO] https://arxiv.org/abs/2306.08647

[38] Yifan Zhou, Sachin Grover, Mohamed El Mistiri, Kamalesh Kalirathnam, Pratyush Kerhalkar, Swaroop Mishra, Neelesh Kumar, Sanket Gaurav, Oya Aran, and Heni Ben Amor. 2025. Prompted Policy Search: Reinforcement Learning through Linguistic and Numerical Reasoning in LLMs. In *Advances in Neural Information Processing Systems*.

[39] Zihao Zhou, Bin Hu, Chenyang Zhao, Pu Zhang, and Bin Liu. 2024. Large Language Model as a Policy Teacher for Training Reinforcement Learning Agents. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*.

[40] Ilya Zisman, Vladislav Kurenkov, Alexander Nikulin, Viacheslav Sinii, and Sergey Kolesnikov. 2024. Emergence of In-Context Reinforcement Learning from Noise Distillation. In *International Conference on Machine Learning*.

Appendix

Behavior-Guided Multimodal Policy Search:
Leveraging Vision-Language Models for Reinforcement Learning

Table of Contents

A Use of Large Language Model **2**

B Extended Disscussion **2**
B.1 Limitation and Future Work . . . 2
B.2 Scaling to Neural-Network Policies . . . 2
B.3 Reproducibility Notes . . . 2

C Technical Lemmas **3**

D Detailed Proofs **3**
D.1 Regret Decomposition . . . 3
D.2 Proof of Theorem 4.4 . . . 5
D.3 Proof of Theorem 4.5 . . . 6

E Experimental Detail **6**
E.1 Problem Taxonomy: Continuous vs. Discrete Spaces . . . 6
E.2 Hyperparameter Configuration for Evolutionary Computing Methods . . . 7
E.3 Hyperparameter Configuration for LLM-Based Methods . . . 7
E.4 Hardware . . . 8

F Additional Experiments **8**
F.1 Effect of Changing the VLM Backbone . . . 8
F.2 Convergence Comparison of LLM-Based Methods . . . 8
F.3 Statistical Significance . . . 9
F.4 Robustness of Per-Environment Comparisons . . . 10

G Detailed Algorithm **11**

H Environment Descriptions **12**
H.1 Discrete-Control Tasks . . . 12
H.2 Continuous-Control Tasks . . . 13

References **14**

A Use of Large Language Model

We did not use large language models to generate the core research ideas, design the method, implement the codebase, or run the experiments in this work. Our use of LLMs was limited to writing assistance during manuscript preparation, specifically for improving grammar, phrasing, and readability, since English is not the authors' native language. All LLM-generated edits were carefully reviewed, verified, and revised by the authors before inclusion in the final manuscript.

B Extended Disscussion

B.1 Limitation and Future Work

Our current formulation of BMPS has several important limitations:

- **High token cost.** Each update must include optimization history, neighbor comparisons, and VLM-generated behavioral feedback, making the method more expensive than reward-only prompt-based optimizers.
- **Context-window constraints.** As trajectories, sampled neighbors, or behavioral descriptions grow, the prompt can approach the context limit of the underlying language model, requiring compression that may discard useful information.
- **Restricted search space.** Policy parameters are currently searched only in the range $[-6, 6]$ and represented with a single decimal digit, which limits search resolution and makes the method difficult to scale to larger neural policies with many parameters.

An important direction for future work is therefore to improve token efficiency, develop more context-aware summarization mechanisms, and design richer parameterizations or hierarchical search spaces that can extend BMPS to larger neural architectures.

B.2 Scaling to Neural-Network Policies

Asking an LLM to generate every parameter of a large neural-network policy does not scale. BMPS avoids this dependence on the full parameter dimension D by searching in a low-dimensional latent subspace of dimension $d \ll D$. At each iteration, we construct an orthonormal random projection matrix $A_t \in \mathbb{R}^{D\times d}$, and the LLM generates only a latent update $z_t \in \mathbb{R}^d$. The full policy is then updated as

$$\theta_{t+1} = \mathrm{clip}\big(\theta_t + \alpha A_t z_t,\ -6,\ 6\big). \tag{B.1}$$

The sizes of the LLM prompt and output therefore depend on d rather than on the potentially much larger number of policy parameters D. Since $A_t^\top A_t = I_d$, the projection preserves the norm of the update before clipping, so α consistently controls the magnitude of the update in the full parameter space.

Table B.1 reports BMPS with multilayer perceptron policies of two sizes, where $[16, 16]$ and $[8, 8]$ denote two-hidden-layer MLPs with 16 and 8 neurons per hidden layer, respectively.

Table B.1: Mean return ± standard deviation of BMPS with MLP policies and latent-subspace search.

MLP hidden layers	Swimmer	Hopper
$[16, 16]$	305.43 ± 45.62	930.56 ± 85.05
$[8, 8]$	318.01 ± 46.15	1150.87 ± 197.05

These preliminary results show that BMPS can operate with neural-network policies and is not restricted to linear ones. Scaling to larger neural-network policies nevertheless remains a limitation shared by this entire class of LLM-based optimizers, including ProPS and ProPS+: whenever the LLM must represent the full parameter vector as text, larger networks incur proportionally larger token costs regardless of the optimization strategy. Latent-subspace search mitigates this cost but does not remove the dependence of search quality on the choice of d and α.

B.3 Reproducibility Notes

The reproducibility of BMPS is necessarily partial because the framework relies on black-box LLM and VLM APIs whose stochastic decoding, backend updates, and service-level variability can introduce non-determinism into both policy search and multimodal diagnosis. Even when prompts, sampling temperatures, anchor-selection rules, and evaluation protocols are fixed, repeated runs may still diverge because the language model may propose different policy updates, the vision-language model may produce slightly different behavioral summaries, and the resulting search trajectory can amplify these differences over multiple iterations.

To support meaningful comparison, we describe the benchmark suite, model choices, training budget, and the key components of the search pipeline in the main paper, and we intend to provide the implementation details needed to reconstruct our experimental setup, including prompts, neighborhood-sampling procedures, anchor definitions, and evaluation settings. Nevertheless, exact replication of every intermediate policy, diagnostic trace, and optimization trajectory should not be expected. We therefore view reproducibility in this setting primarily as reproducing the experimental protocol and the aggregate performance trends, rather than reproducing an identical sequence of model outputs at every step.

C Technical Lemmas

This appendix collects the tools used in the proofs of Section 4. All of them are standard, and we state them without proof, with references. Throughout, $D_{\mathrm{KL}}(Q\|\Pi) = \mathbb{E}_Q[\log(dQ/d\Pi)]$ if $Q \ll \Pi$ and $D_{\mathrm{KL}}(Q\|\Pi) = +\infty$ otherwise. We write $(x)_+ = \max\{x, 0\}$. All functions of θ that appear in expectations (such as j) are assumed to be measurable; this holds trivially when Θ is countable.

Lemma C.1 (Hoeffding's inequality [4]). *Let $Y_1, \dots, Y_n$ be independent random variables with $Y_i \in [a_i, b_i]$ almost surely, and let $S = \sum_{i=1}^n Y_i$. For every $\varepsilon > 0$,*

$$\Pr\big(S - \mathbb{E}S \geq \varepsilon\big) \leq \exp\Big(-\frac{2\varepsilon^2}{\sum_{i=1}^n (b_i - a_i)^2}\Big),$$

and the same bound holds for $\Pr(S - \mathbb{E}S \leq -\varepsilon)$.

For a random variable θ^* with values in a finite set C and an event A with $\Pr(A) > 0$, we write

$$H(\theta^* \mid A) = -\sum_{\theta \in C} \Pr(\theta^* = \theta \mid A) \log \Pr(\theta^* = \theta \mid A)$$

for the entropy of the conditional distribution of θ^* given A, with $0 \log 0 = 0$. For a random variable Y with countably many values, we write

$$H(\theta^* \mid A, Y) = \sum_y \Pr(Y = y \mid A)\, H\big(\theta^* \mid A \cap \{Y = y\}\big),$$

where the sum runs over the values y with $\Pr(A, Y = y) > 0$, and we drop A from the notation when it is the sure event.

Lemma C.2 (Conditioning reduces entropy [1]). *Let θ^* take values in a finite set C, let A be an event with $\Pr(A) > 0$, and let Y be a random variable with countably many values. Then*

$$0 \leq H(\theta^* \mid A, Y) \leq H(\theta^* \mid A) \leq \log |C|.$$

For distributions P and P' on a countable set, $D_{\mathrm{KL}}(P\|P')$ is the KL divergence defined at the beginning of this appendix, and $\mathrm{TV}(P, P') = \sup_E |P(E) - P'(E)|$ is the total variation distance. For an event A with $\Pr(A) > 0$ and a random variable Y with countably many values, $P_{Y|A}$ denotes the conditional distribution of Y given A.

Lemma C.3 (Mutual information, data processing, and Pinsker's inequality [1]). *Let θ^* take values in a finite set C, let A be an event with $\Pr(A) > 0$, let $\pi(\theta) = \Pr(\theta^* = \theta \mid A)$, and let Y and Y' be random variables with countably many values such that Y is a function of Y'. (a) $H(\theta^* \mid A) - H(\theta^* \mid A, Y) = \sum_{\theta:\pi(\theta)>0} \pi(\theta)\, D_{\mathrm{KL}}\big(P_{Y|A\cap\{\theta^*=\theta\}} \big\| P_{Y|A}\big)$. (b) $H(\theta^* \mid A, Y') \leq H(\theta^* \mid A, Y)$. (c) For distributions P and P' on a countable set, every map ϕ on that set, and every f with values in $[0, 1]$, $D_{\mathrm{KL}}(P \circ \phi^{-1}\|P' \circ \phi^{-1}) \leq D_{\mathrm{KL}}(P\|P')$ and $|\mathbb{E}_P[f] - \mathbb{E}_{P'}[f]| \leq \mathrm{TV}(P, P') \leq \sqrt{D_{\mathrm{KL}}(P\|P')/2}$.*

D Detailed Proofs

This appendix states and proves the regret decomposition behind Section 4.1 and proves the theorems of Section 4. Notation follows Sections 3 and 4.

D.1 Regret Decomposition

A scalar return may leave the behavioral cause of an outcome ambiguous: two low-return locomotion policies may fail because of oscillation and because of insufficient motion, respectively. A description can distinguish the two cases, but this matters for search only if it changes which policy is worth proposing. We formalize this point with a decision-theoretic reference model.

Under Assumption 4.2, for values x and ψ of positive probability, we define the return assessments of an ideal decision maker without and with the descriptions,

$$\begin{aligned} m_{0,t}(\theta; x) &= \mathbb{E}\big[j_\xi(\theta) \mid X_t = x\big], \\ m_{1,t}(\theta; x, \psi) &= \mathbb{E}\big[j_\xi(\theta) \mid X_t = x, \Psi_t = \psi\big], \end{aligned} \tag{D.1}$$

the values of proposing the best policy under each assessment,

$$\begin{aligned} V_{0,t}(x) &= \max_{\theta \in C} m_{0,t}(\theta; x), \\ V_{1,t}(x) &= \mathbb{E}_{\Psi_t|x}\Big[\max_{\theta \in C} m_{1,t}(\theta; x, \Psi_t)\Big], \end{aligned} \tag{D.2}$$

the *decision value* $\mathcal{I}_t = V_{1,t} - V_{0,t}$ of the descriptions, and the *residual uncertainty* $\mathcal{U}_t(x) = \mathbb{E}[j_\xi(\theta^*) \mid X_t = x] - V_{0,t}(x)$. With $P(\cdot \mid x) = P_{\mathrm{LLM}}(\cdot \mid x)$ and $Q(\cdot \mid x, \psi) = P_{\mathrm{LLM}}(\cdot \mid x, \psi)$, the *proposal regrets* of the actual proposals are

$$\begin{aligned} \mathcal{R}_{0,t}(x) &= V_{0,t}(x) - \mathbb{E}_{\theta \sim P(\cdot|x)}\big[m_{0,t}(\theta; x)\big], \\ \mathcal{R}_{1,t}(x) &= V_{1,t}(x) \\ &\quad - \mathbb{E}_{\Psi_t|x}\, \mathbb{E}_{\theta \sim Q(\cdot|x,\Psi_t)}\big[m_{1,t}(\theta; x, \Psi_t)\big]. \end{aligned} \tag{D.3}$$

Proposition D.1 (Regret decomposition). *Under Assumptions 4.1 and 4.2, for every t and every x with $\Pr(X_t = x) > 0$, the following hold.*

(a) $0 \le \mathcal{I}_t(x) \le \mathcal{U}_t(x)$. *If θ_0^* maximizes $m_{0,t}(\cdot; x)$, then $\mathcal{I}_t(x) > 0$ if and only if, with positive conditional probability given $X_t = x$, the descriptions make θ_0^* strictly suboptimal under $m_{1,t}$.*

(b) *If $\theta_t^P \sim P_{\mathrm{LLM}}(\cdot \mid X_t)$ is drawn by a separate query with the numerical prompt and fresh decoding randomness, then $\mathbb{E}[j_\xi(\theta_t) - j_\xi(\theta_t^P) \mid X_t = x] = \mathcal{I}_t(x) - \mathcal{R}_{1,t}(x) + \mathcal{R}_{0,t}(x)$.*

(c) *The Bayesian regret of the proposals satisfies*

$$\begin{aligned} \mathrm{BR}_T &= \mathbb{E}\Big[\sum_{t=1}^{T} \big(j_\xi(\theta^*) - j_\xi(\theta_t)\big)\Big] \\ &= \sum_{t=1}^{T} \mathbb{E}\big[\mathcal{U}_t(X_t) - \mathcal{I}_t(X_t) + \mathcal{R}_{1,t}(X_t)\big]. \end{aligned} \tag{D.4}$$

Behavioral feedback is therefore valuable exactly when it can overturn the decision that the numerical evidence supports; a vivid description that leaves the ranking of candidates unchanged has zero decision value. Part (b) shows that the feedback improves an actual proposal exactly when its decision value exceeds the increase in proposal regret that it causes: the VLM determines the available information $\mathcal{I}_t$, and the LLM determines how much of it is lost through $\mathcal{R}_{1,t}$. Part (c) shows that the same two quantities govern the cumulative regret. At every iteration, the regret is the uncertainty left by the numerical context, minus what the descriptions resolve for the decision, plus what the LLM loses in using them. Since $\mathcal{U}_t - \mathcal{I}_t \ge 0$, even an ideal proposer incurs regret until the record resolves the decision-relevant uncertainty.

Proof. Fix t and x with $\Pr(X_t = x) > 0$. To lighten notation, we drop the subscript t and write X, Ψ, m_0, m_1, V_0, V_1, $\mathcal{I}$, $\mathcal{U}$, $\mathcal{R}_0$, and $\mathcal{R}_1$. Let $\mathcal{S}_x = \{\psi : \Pr(X = x, \Psi = \psi) > 0\}$, which is countable, and write $w_x(\psi) = \Pr(\Psi = \psi \mid X = x)$ for $\psi \in \mathcal{S}_x$, so that $\sum_{\psi \in \mathcal{S}_x} w_x(\psi) = 1$. Since X and Ψ take countably many values, every conditioning event below has positive probability, and conditional expectations are expectations under the elementary conditional distributions $\Pr(\cdot \mid X = x)$ and $\Pr(\cdot \mid X = x, \Psi = \psi)$.

Part (a). The event $\{X = x\}$ is the disjoint union of the events $\{X = x, \Psi = \psi\}$, and those with $\psi \notin \mathcal{S}_x$ have probability zero. For each $\theta \in C$, the random variable $j_\xi(\theta)$ takes values in $[0, 1]$, so the law of total expectation under $\Pr(\cdot \mid X = x)$ gives

$$\sum_{\psi \in \mathcal{S}_x} w_x(\psi)\, m_1(\theta; x, \psi) = \sum_{\psi \in \mathcal{S}_x} \Pr(\Psi = \psi \mid X = x)\, \mathbb{E}\big[j_\xi(\theta) \mid X = x, \Psi = \psi\big] = \mathbb{E}\big[j_\xi(\theta) \mid X = x\big] = m_0(\theta; x). \tag{D.5}$$

For every $\theta' \in C$ and $\psi \in \mathcal{S}_x$, we have $\max_{\theta \in C} m_1(\theta; x, \psi) \ge m_1(\theta'; x, \psi)$. Multiplying by $w_x(\psi)$, summing over ψ, and using Eq. (D.5) gives $V_1(x) \ge m_0(\theta'; x)$, and maximizing over θ' yields $\mathcal{I}(x) \ge 0$. Similarly, $j_\xi(\theta) \le j_\xi(\theta^*)$ gives $m_1(\theta; x, \psi) \le \mathbb{E}[j_\xi(\theta^*) \mid X = x, \Psi = \psi]$ for every $\theta \in C$ and $\psi \in \mathcal{S}_x$. Maximizing over θ, averaging over ψ with the weights $w_x(\psi)$, and applying the law of total expectation yields $V_1(x) \le \mathbb{E}[j_\xi(\theta^*) \mid X = x]$, that is, $\mathcal{I}(x) \le \mathcal{U}(x)$.

Now let θ_0^* maximize $m_0(\cdot; x)$, which is possible because C is finite. Then $V_0(x) = m_0(\theta_0^*; x) = \sum_{\psi \in \mathcal{S}_x} w_x(\psi)\, m_1(\theta_0^*; x, \psi)$ by Eq. (D.5). Subtracting this series from the series for $V_1(x)$, both of which converge absolutely, gives

$$\mathcal{I}(x) = \sum_{\psi \in \mathcal{S}_x} w_x(\psi)\, \Delta_x(\psi), \qquad \Delta_x(\psi) = \max_{\theta \in C} m_1(\theta; x, \psi) - m_1(\theta_0^*; x, \psi).$$

Each $\Delta_x(\psi)$ is nonnegative, and each weight $w_x(\psi)$ with $\psi \in \mathcal{S}_x$ is positive. Hence $\mathcal{I}(x) > 0$ if and only if $\Delta_x(\psi) > 0$ for some $\psi \in \mathcal{S}_x$, that is, if and only if $\Pr(\Delta_x(\Psi) > 0 \mid X = x) > 0$. The event $\Delta_x(\psi) > 0$ means that θ_0^* is strictly suboptimal under the assessment that includes the descriptions ψ.

Part (b). Write $\theta^Q = \theta_t$ and $\theta^P = \theta_t^P$. By Assumption 4.1, θ^Q is conditionally independent of ξ given (X, Ψ), and $\Pr(\theta^Q = \theta \mid X = x, \Psi = \psi) = Q(\theta \mid x, \psi)$. Because θ^P is drawn by a separate query with the numerical prompt and fresh decoding randomness, θ^P is conditionally independent of ξ given X, and $\Pr(\theta^P = \theta \mid X = x) = P(\theta \mid x)$. Since θ^P takes values in C, the events $\{\theta^P = \theta\}$, $\theta \in C$, partition the sample space, and $j_\xi(\theta^P)$ coincides with $j_\xi(\theta)$ on $\{\theta^P = \theta\}$; the same holds for θ^Q. Whenever $\Pr(X = x, \theta^P = \theta) > 0$, conditional independence means that the conditional distribution of ξ given $(X = x, \theta^P = \theta)$ equals that given $X = x$, so $\mathbb{E}[j_\xi(\theta) \mid X = x, \theta^P = \theta] = m_0(\theta; x)$. The law of total expectation over the partition $\{\theta^P = \theta\}$ therefore gives

$$\mathbb{E}\big[j_\xi(\theta^P) \mid X = x\big] = \sum_{\theta \in C} P(\theta \mid x)\, m_0(\theta; x) = V_0(x) - \mathcal{R}_0(x).$$

The same argument with (X, Ψ) in place of X gives $\mathbb{E}[j_\xi(\theta^Q) \mid X = x, \Psi = \psi] = \sum_{\theta \in C} Q(\theta \mid x, \psi)\, m_1(\theta; x, \psi)$ for every $\psi \in \mathcal{S}_x$. Averaging over ψ with the weights $w_x(\psi)$, as in Eq. (D.5), yields

$$\mathbb{E}\big[j_\xi(\theta^Q) \mid X = x\big] = \sum_{\psi \in \mathcal{S}_x} w_x(\psi)\, \mathbb{E}_{\theta \sim Q(\cdot \mid x, \psi)}\big[m_1(\theta; x, \psi)\big] = V_1(x) - \mathcal{R}_1(x).$$

Subtracting the two expressions gives $\mathbb{E}[j_\xi(\theta^Q) - j_\xi(\theta^P) \mid X = x] = \mathcal{I}(x) - \mathcal{R}_1(x) + \mathcal{R}_0(x)$.

Part (c). By the definition of $\mathcal{U}$, $\mathbb{E}[j_\xi(\theta^*) \mid X = x] = V_0(x) + \mathcal{U}(x)$. Subtracting $\mathbb{E}[j_\xi(\theta_t) \mid X = x] = V_1(x) - \mathcal{R}_1(x)$ from part (b) gives

$$\mathbb{E}\big[j_\xi(\theta^*) - j_\xi(\theta_t) \mid X_t = x\big] = \mathcal{U}_t(x) - \mathcal{I}_t(x) + \mathcal{R}_{1,t}(x).$$

The random variable $j_\xi(\theta^*) - j_\xi(\theta_t)$ takes values in $[0,1]$, and X_t takes countably many values, so the law of total expectation over the events $\{X_t = x\}$ of positive probability gives

$$\mathbb{E}\big[j_\xi(\theta^*) - j_\xi(\theta_t)\big] = \sum_x \Pr(X_t = x)\,\mathbb{E}\big[j_\xi(\theta^*) - j_\xi(\theta_t) \mid X_t = x\big] = \mathbb{E}\big[\mathcal{U}_t(X_t) - \mathcal{I}_t(X_t) + \mathcal{R}_{1,t}(X_t)\big].$$

Summing over $t = 1, \dots, T$ proves Eq. (D.4). □

D.2 Proof of Theorem 4.4

Proof. The first two steps follow [7].

Step 1: regret as a sum of conditional regrets. Since $\theta_t \in C$, the random variable $j_\xi(\theta^*) - j_\xi(\theta_t)$ takes values in $[0,1]$. Because $Z_{1:t}$ takes countably many values, the law of total expectation gives

$$\mathbb{E}\big[j_\xi(\theta^*) - j_\xi(\theta_t)\big] = \sum_{z_{1:t}} \Pr(Z_{1:t} = z_{1:t})\,\varrho_t(z_{1:t}) = \mathbb{E}\big[\varrho_t(Z_{1:t})\big],$$

where the sum runs over the records of positive probability. Summing over t yields $\mathrm{BR}_T = \sum_{t=1}^T \mathbb{E}[\varrho_t(Z_{1:t})]$.

Step 2: the information gains telescope. By Lemma C.2, applied with $A = \{Z_{1:t} = z_{1:t}\}$ and $Y = Z_{t+1}$, we have $\iota_t(z_{1:t}) \geq 0$. By the definition of $H(\theta^* \mid A, Y)$,

$$\begin{aligned}\mathbb{E}\big[\iota_t(Z_{1:t})\big] &= \sum_{z_{1:t}} \Pr(Z_{1:t} = z_{1:t})\, H(\theta^* \mid Z_{1:t} = z_{1:t}) \\ &\quad - \sum_{z_{1:t}} \Pr(Z_{1:t} = z_{1:t}) \sum_z \Pr(Z_{t+1} = z \mid Z_{1:t} = z_{1:t})\, H\big(\theta^* \mid Z_{1:t+1} = (z_{1:t}, z)\big) \\ &= H(\theta^* \mid Z_{1:t}) - H(\theta^* \mid Z_{1:t+1}),\end{aligned}$$

because $\Pr(Z_{1:t} = z_{1:t})\Pr(Z_{t+1} = z \mid Z_{1:t} = z_{1:t}) = \Pr(Z_{1:t+1} = (z_{1:t}, z))$. All entropies lie in $[0, \log|C|]$, so these series converge. Summing over t and using Lemma C.2 once more,

$$\sum_{t=1}^T \mathbb{E}\big[\iota_t(Z_{1:t})\big] = H(\theta^* \mid Z_1) - H(\theta^* \mid Z_{1:T+1}) \leq H(\theta^* \mid Z_1) \leq \log|C|. \tag{D.6}$$

Fix t and a record $z_{1:t}$ of positive probability, let $A = \{Z_{1:t} = z_{1:t}\}$, $\pi = \pi_t$, $Q = Q_t = P_{\mathrm{LLM}}(\cdot \mid z_t)$, and $m(\theta) = \mathbb{E}[j_\xi(\theta) \mid A]$.

Step 3: comparison with posterior sampling. By Assumption 4.1, conditionally on Z_t, the proposal θ_t is independent of ξ and of $Z_{1:t-1}$, which are generated before the query. Hence θ_t is conditionally independent of ξ given $Z_{1:t}$, and its conditional distribution given A is Q. The argument of part (b) of the proof of Proposition D.1, with A in place of $\{X = x\}$, gives $\mathbb{E}[j_\xi(\theta_t) \mid A] = \sum_\theta Q(\theta)\, m(\theta)$. Therefore

$$\varrho_t(z_{1:t}) = \varrho_t^\pi(z_{1:t}) + \sum_{\theta \in C} \big(\pi(\theta) - Q(\theta)\big)\, m(\theta) \leq \varrho_t^\pi(z_{1:t}) + \varepsilon_t(z_{1:t}),$$

where the inequality follows from Lemma C.3(c), because m takes values in $[0,1]$ and $\mathrm{TV}(Q, \pi) \leq \varepsilon_t(z_{1:t})$.

Step 4: information of the actual proposal. Since $Q(\theta) \geq \pi(\theta)/c$, every θ with $\pi(\theta) > 0$ satisfies $\Pr(A, \theta_t = \theta) > 0$, so $\iota_t(\theta; z_{1:t})$ is defined for these θ, and it is nonnegative by Lemma C.2. The condition of the theorem then gives

$$\varrho_t^\pi(z_{1:t}) \leq \sqrt{\Gamma^\circ \sum_\theta \pi(\theta)\, \iota_t(\theta; z_{1:t})} \leq \sqrt{c\,\Gamma^\circ \sum_\theta Q(\theta)\, \iota_t(\theta; z_{1:t})}.$$

We claim that $\sum_\theta Q(\theta)\, \iota_t(\theta; z_{1:t}) = \iota_t(z_{1:t})$. Since Z_{t+1} records θ_t, grouping the values of Z_{t+1} according to the value of θ_t gives $H(\theta^* \mid A, Z_{t+1}) = \sum_\theta Q(\theta)\, H(\theta^* \mid A_\theta, Z_{t+1})$, where the sum runs over the θ with $Q(\theta) > 0$. Moreover, θ^* is a function of ξ, so θ_t is conditionally independent of θ^* given A, and $\Pr(\theta^* = \cdot \mid A_\theta) = \pi$, which gives $H(\theta^* \mid A_\theta) = H(\theta^* \mid A)$. Combining the two identities,

$$\iota_t(z_{1:t}) = \sum_\theta Q(\theta)\big(H(\theta^* \mid A_\theta) - H(\theta^* \mid A_\theta, Z_{t+1})\big) = \sum_\theta Q(\theta)\, \iota_t(\theta; z_{1:t}).$$

Step 5: summation. Steps 3 and 4 give $\varrho_t(z_{1:t}) \leq \sqrt{c\,\Gamma^\circ\, \iota_t(z_{1:t})} + \varepsilon_t(z_{1:t})$ for every record. By Step 1, the Cauchy–Schwarz inequality, and Eq. (D.6),

$$\mathrm{BR}_T = \sum_{t=1}^T \mathbb{E}\big[\varrho_t(Z_{1:t})\big] \leq \sqrt{c\,\Gamma^\circ} \sqrt{T \sum_{t=1}^T \mathbb{E}\big[\iota_t(Z_{1:t})\big]} + \sum_{t=1}^T \mathbb{E}\big[\varepsilon_t(Z_{1:t})\big] \leq \sqrt{c\,\Gamma^\circ\, T \log|C|} + \sum_{t=1}^T \mathbb{E}\big[\varepsilon_t(Z_{1:t})\big],$$

which is Eq. (7).

Step 6: group feedback. Fix a record and use the notation of Step 3. For θ' with $\pi(\theta') > 0$, let $\mu(\theta \mid \theta') = \mathbb{E}[j_\xi(\theta) \mid A \cap \{\theta^* = \theta'\}]$. The law of total expectation over the events $\{\theta^* = \theta\}$ gives $\mathbb{E}[j_\xi(\theta^*) \mid A] = \sum_\theta \pi(\theta)\, \mu(\theta \mid \theta)$, so

$$\varrho_t^\pi(z_{1:t}) = \sum_{\theta : \pi(\theta) > 0} \pi(\theta)\big(\mu(\theta \mid \theta) - m(\theta)\big).$$

Let G be a group with $\pi(G) = \sum_{\theta\in G}\pi(\theta) > 0$, let $\theta'' \in G$ satisfy $\pi(\theta'') > 0$, and let $Y_G = (Y(\theta))_{\theta\in G}$ be the statistics that Z_{t+1} contains when $\theta_t = \theta''$; Y_G is a function of Z_{t+1}. By Step 3, ξ has the same conditional distribution given $A_{\theta''}$ as given A, and so does θ^*. Since each $Y(\theta)$ has conditional mean $j_\xi(\theta)$ given ξ and the past, $\mathbb{E}[Y(\theta) \mid A_{\theta''} \cap \{\theta^* = \theta'\}] = \mu(\theta \mid \theta')$ and $\mathbb{E}[Y(\theta) \mid A_{\theta''}] = m(\theta)$. For $\theta \in G$ with $\pi(\theta) > 0$, Lemma C.3(c), applied first to the conditional distributions of $Y(\theta)$ and then to the projection from Y_G to $Y(\theta)$, gives

$$\big(\mu(\theta \mid \theta) - m(\theta)\big)^2 \le \frac{1}{2} D_{\mathrm{KL}}\big(P_{Y(\theta)|A_{\theta''}\cap\{\theta^*=\theta\}} \big\| P_{Y(\theta)|A_{\theta''}}\big) \le \frac{1}{2} D_{\mathrm{KL}}\big(P_{Y_G|A_{\theta''}\cap\{\theta^*=\theta\}} \big\| P_{Y_G|A_{\theta''}}\big).$$

Multiplying by $\pi(\theta) = \Pr(\theta^* = \theta \mid A_{\theta''})$, summing over $\theta \in G$, extending the sum to all of C, and applying Lemma C.3(a) and (b) give

$$\sum_{\theta\in G} \pi(\theta)\big(\mu(\theta \mid \theta) - m(\theta)\big)^2 \le \frac{1}{2}\Big(H(\theta^* \mid A_{\theta''}) - H(\theta^* \mid A_{\theta''}, Y_G)\Big) \le \frac{1}{2}\, \iota_t(\theta''; z_{1:t}).$$

Averaging over $\theta'' \in G$ with the weights $\pi(\theta'')/\pi(G)$ yields $\sum_{\theta\in G}\pi(\theta)(\mu(\theta \mid \theta) - m(\theta))^2 \le \frac{1}{2\pi(G)} \sum_{\theta''\in G} \pi(\theta'')\, \iota_t(\theta''; z_{1:t})$. The Cauchy–Schwarz inequality within each group and then across the at most M groups with $\pi(G) > 0$ gives

$$\varrho_t^\pi(z_{1:t}) \le \sum_G \sqrt{\pi(G)} \sqrt{\sum_{\theta\in G} \pi(\theta)\big(\mu(\theta \mid \theta) - m(\theta)\big)^2} \le \sum_G \sqrt{\frac{1}{2}\sum_{\theta''\in G} \pi(\theta'')\, \iota_t(\theta''; z_{1:t})} \le \sqrt{\frac{M}{2}\sum_\theta \pi(\theta)\, \iota_t(\theta; z_{1:t})}.$$

If $\varrho_t^\pi(z_{1:t}) \ge 0$, squaring gives the condition with $\Gamma^\circ = M/2$; otherwise the condition holds trivially. □

D.3 Proof of Theorem 4.5

Proof. Let $\mathcal{F}_{t-1}$ contain everything observed before the LLM is queried at iteration t. Then $q_t = Q_t(\mathcal{S}_\varepsilon)$ is determined by $\mathcal{F}_{t-1}$, and Assumption 4.1 gives $\Pr(\theta_t \in \mathcal{S}_\varepsilon \mid \mathcal{F}_{t-1}) = q_t$.

Step 1: some proposal is ε-optimal. Let E_t be the event that none of $\theta_1, \dots, \theta_t$ lies in $\mathcal{S}_\varepsilon$, and let E_0 be the sure event. Define $M_t = \exp(\sum_{i\le t} q_i)$ on E_t and $M_t = 0$ outside E_t, so that $M_0 = 1$. The event E_{t-1} and the numbers $q_1, \dots, q_t$ are determined by $\mathcal{F}_{t-1}$. On E_{t-1}, since $E_t = E_{t-1} \cap \{\theta_t \notin \mathcal{S}_\varepsilon\}$,

$$\mathbb{E}[M_t \mid \mathcal{F}_{t-1}] = \exp\Big(\sum_{i\le t} q_i\Big) \Pr\big(\theta_t \notin \mathcal{S}_\varepsilon \mid \mathcal{F}_{t-1}\big) = M_{t-1}\, e^{q_t}(1 - q_t) \le M_{t-1},$$

by $1 - u \le e^{-u}$; outside E_{t-1}, $M_t = M_{t-1} = 0$. Taking expectations and iterating gives $\mathbb{E}[M_T] \le 1$. On $E_T \cap \{\sum_t q_t \ge s\}$, we have $M_T \ge e^s$, so Markov's inequality gives $\Pr(E_T \cap \{\sum_t q_t \ge s\}) \le e^{-s}$.

Step 2: the recorded returns are accurate. Let $\varepsilon_L = \sqrt{\log(2K/\delta)/(2L)}$ and let $\vartheta_1, \dots, \vartheta_K$ be the evaluated policies in the order of evaluation, with recorded normalized returns $\widehat{j}(\vartheta_k)$. Conditionally on ϑ_k and on everything observed before its evaluation, the L normalized returns that define $\widehat{j}(\vartheta_k)$ are independent, take values in $[0, 1]$, and have mean $j(\vartheta_k)$. The two-sided form of Lemma C.1 bounds the conditional probability of $|\widehat{j}(\vartheta_k) - j(\vartheta_k)| > \varepsilon_L$ by $2\exp(-2L\varepsilon_L^2) = \delta/K$, and taking expectations bounds the unconditional probability by the same quantity. A union bound over k shows that the event $\mathcal{E}' = \{|\widehat{j}(\vartheta_k) - j(\vartheta_k)| \le \varepsilon_L \text{ for all } k\}$ has probability at least $1 - \delta$.

Step 3: conclusion. Suppose that $\mathcal{E}'$ holds and E_T fails, so that some $\theta_{t'}$ lies in $\mathcal{S}_\varepsilon$. Since $\widehat{\theta}_T$ maximizes $\widehat{j}$ over a set that contains $\theta_{t'}$,

$$j(\widehat{\theta}_T) \ge \widehat{j}(\widehat{\theta}_T) - \varepsilon_L \ge \widehat{j}(\theta_{t'}) - \varepsilon_L \ge j(\theta_{t'}) - 2\varepsilon_L \ge j(\theta^*) - \varepsilon - 2\varepsilon_L,$$

which is Eq. (8). Hence the event that $\sum_t q_t \ge s$ and Eq. (8) fails is contained in $(E_T \cap \{\sum_t q_t \ge s\}) \cup (\mathcal{E}')^c$, whose probability is at most $e^{-s} + \delta$ by Steps 1 and 2.

Finally, $j(\theta^*) - j(\theta) \ge 0$ for every θ, so Markov's inequality under Q_t gives $1 - q_t = Q_t(\{\theta : j(\theta^*) - j(\theta) > \varepsilon\}) \le \mathbb{E}_{Q_t}[j(\theta^*) - j]/\varepsilon = r_t/\varepsilon$. □

E Experimental Detail

E.1 Problem Taxonomy: Continuous vs. Discrete Spaces

We categorize environments along two axes-state space and action space-because the choice of policy parameterization and search strategy depends critically on this structure.

Continuous state, continuous action. Both EC and LLM-guided methods represent the policy as a linear mapping $\mathbf{a} = \mathbf{W}\mathbf{s} + \mathbf{b}$. The output $\mathbf{a} \in \mathbb{R}^{d_a}$ is applied directly to the environment without discretization. This setup is standard for continuous-control tasks. The only difference between EC and LLM methods lies in how they update the parameters: EC uses evolutionary perturbations (CMA-ES, ARS, etc.), while LLM methods treat the flattened parameter vector as a black-box function and use the LLM's in-context optimization to propose the next θ.

Continuous state, discrete action. The state remains a real-valued vector, but the action space is finite (e.g., Pong with 3 actions). Both EC and LLM methods still use a linear policy $\mathbf{W}\mathbf{s} + \mathbf{b}$, but the output is interpreted as a vector of *logits*. The discrete action is selected via arg max over these logits. Thus, the policy architecture is identical to the continuous-action case; only the action-selection rule changes. This bridges continuous observation spaces to discrete decision spaces seamlessly.

Discrete state, discrete action. Here the state is an integer index (e.g., $s \in \{0, \ldots, 47\}$ for CliffWalking). The policy representations diverge between EC and LLM methods:

- **EC methods** search over a global tabular representation. We consider a Q-value tabular parameterization, where $\theta \in \mathbb{R}^{|S|\times|A|}$ and the induced policy is $\pi(s) = \arg\max_a \theta[s, a]$.
- **LLM methods** also search a global tabular representation. For small discrete environments, the LLM proposes a flat parameter vector encoding either a state→action lookup table (Q-table) or the flattened Q-value matrix. During rollout, the policy simply indexes into this table. The LLM does *not* query the model at every timestep; rather, it proposes the *entire* parameter vector once per training iteration, using the reward history from previous iterations as context.

E.2 Hyperparameter Configuration for Evolutionary Computing Methods

All evolutionary computing (EC) baselines share identical hyperparameters across environments.

Augmented Random Search (ARS).. ARS [6] approximates policy gradients via finite differences: it samples N directions $\delta_i \sim \mathcal{N}(0, I)$, evaluates symmetric perturbations $\theta \pm \nu\delta_i$, and updates using the top-k directions by reward difference. Reward normalization by empirical std stabilizes learning across heterogeneous scales. ARS is memory-efficient and highly parallelizable, making it suitable for high-dimensional linear policies.

Hyperparameter	Value
n_directions	32
deltas_used	16
step_size	0.03
delta_std	0.1
shift	0.0
reward_normalization_epsilon	10^{-8}
grad_batch_size	500

Covariance Matrix Adaptation ES (CMA-ES).. CMA-ES [3] adapts a covariance matrix $\mathbf{C}_t$ to model parameter correlations, enabling efficient search on ill-conditioned landscapes. The distribution $\mathcal{N}(\mathbf{m}_t, \sigma_t^2\mathbf{C}_t)$ is updated via evolution paths for automatic step-size control. We use `pycma` with `covariance_type = auto` (full covariance if dim $\leq$ 256, else diagonal) to balance expressiveness and cost. Robust to noise and gradient-free.

Hyperparameter	Value
population_size	32
sigma	0.1
elite_count	null (auto)
covariance_type	auto
full_covariance_max_dim	256
min_sigma	10^{-12}

OpenAI-ES.. OpenAI-ES [8] uses antithetic sampling ($\theta \pm \sigma\delta_i$) to reduce gradient variance and centered-rank normalization for robustness to reward outliers. The estimated gradient $\hat{g} = \frac{1}{\lambda\sigma}\sum r'_i\delta_i$ is optimized via Adam, combining ES exploration with adaptive learning rates. Highly parallelizable and effective for moderate-dimensional continuous policies.

Hyperparameter	Value
population_size	32
noise_stdev	0.1
learning_rate	0.03
return_proc_mode	centered_rank
optimizer	adam ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$)
weight_decay	0.0
grad_batch_size	500

(μ, λ)-ES.. Classic generational ES where μ parents produce λ offspring via BLX-α crossover and Gaussian mutation. Offspring parameters are sampled from an expanded interval around parent values, promoting diversity. Comma selection (only offspring compete) encourages exploration. Hyperparameters balance exploration ($\alpha = 0.5$, `mutpb`=0.3) and exploitation (`cxpb`=0.6), providing a strong baseline for discrete and low-dimensional tasks.

E.3 Hyperparameter Configuration for LLM-Based Methods

Table E.1 summarizes the shared configuration used by the LLM-based methods in our experiments. Unless otherwise noted, ProPS, ProPS+, and BMPS use the same optimizer LLM and decoding settings, and differ primarily in the information included in the prompt.

Hyperparameter	Value
mu, lam	16, 32
sigma	0.1
sigma_decay, min_sigma	1.0, 10^{-12}
param_bound	10.0
cxmode, alpha	blend, 0.5
cxpb, mutpb	0.6, 0.3
smin, smax	0.01, 0.5
clip	True

Table E.1: Configuration of LLM-based methods.

Setting	ProPS	ProPS+	BMPS
Optimizer LLM	GPT-OSS-120B	GPT-OSS-120B	GPT-OSS-120B
LLM temperature	1.0	1.0	1.0
Optimization history	Yes	Yes	Yes
Environment prior in prompt	No	Yes	Yes
Behavioral visual feedback	No	No	Yes
Diagnostic VLM	-	-	Gemma 3 27B-IT
VLM temperature	-	-	1.0
Anchor policies	-	-	Current / Best / Worst
Neighbor sampling	-	-	Yes
Neighbors per anchor	-	-	5
Interaction budget	8×10^6 environment steps		

E.4 Hardware

All experiments were conducted on a machine equipped with an Intel(R) Core(TM) i7-8700 CPU @ 3.20 GHz (6 cores, 12 threads), 32GB RAM, and no GPU acceleration.

F Additional Experiments

F.1 Effect of Changing the VLM Backbone

We also study how sensitive BMPS is to the choice of the vision-language model used for behavioral diagnosis. In our main experiments, we use Gemma 3 27B-IT as the default VLM. As an additional comparison, we replace this VLM with Gemini 2.5 Flash while keeping the rest of the pipeline fixed, including the optimizer LLM, prompt structure, and evaluation protocol. This experiment isolates the effect of the visual diagnostic backbone itself.

Changing the VLM can influence performance through the quality and consistency of the behavioral summaries extracted from representative frames. A stronger VLM may provide more precise descriptions of failure modes, spatial relations, and trajectory-level behavior, while a weaker or less stable model may introduce noisier feedback that makes policy updates less reliable across iterations. This comparison is therefore useful for understanding whether the gains of BMPS are robust to the choice of multimodal diagnostic model and how portable the framework is across different VLM backbones.

Table F.1 reports the performance of BMPS when the diagnostic VLM is changed from Gemma 3 27B-IT to Gemini 2.5 Flash on the subset of environments for which this comparison was available. Overall, the results suggest that BMPS remains effective across different VLM backbones, while the exact return and variance can vary across tasks depending on the quality and stability of the generated behavioral summaries.

Table F.1: Effect of changing the VLM backbone in BMPS (completed runs only). "BMPS (Gemini 2.5 Flash)" uses Gemini 2.5 Flash as the diagnostic VLM, while "BMPS (Gemma 3 27B-IT)" uses the default Gemma 3 27B-IT backbone. Higher values are better; for negative rewards, less negative is better.

Problem	BMPS (Gemini 2.5 Flash)	BMPS (Gemma 3 27B-IT)
MountainCarContinuous	99.07 ± 0.15	**99.16 ± 0.12**
MountainCar	-117.06 ± 16.25	**-112.01 ± 2.85**
Reacher	**-7.28 ± 0.38**	-7.36 ± 0.66
Swimmer	**320.74 ± 27.49**	309.36 ± 51.83
FrozenLake	0.965 ± 0.023	0.965 ± 0.023
CliffWalking	**-58.31 ± 16.21**	-66.86 ± 8.47
Acrobot	-75.42 ± 2.72	-75.42 ± 2.72
LunarLander	73.24 ± 145.46	**104.83 ± 168.86**

F.2 Convergence Comparison of LLM-Based Methods

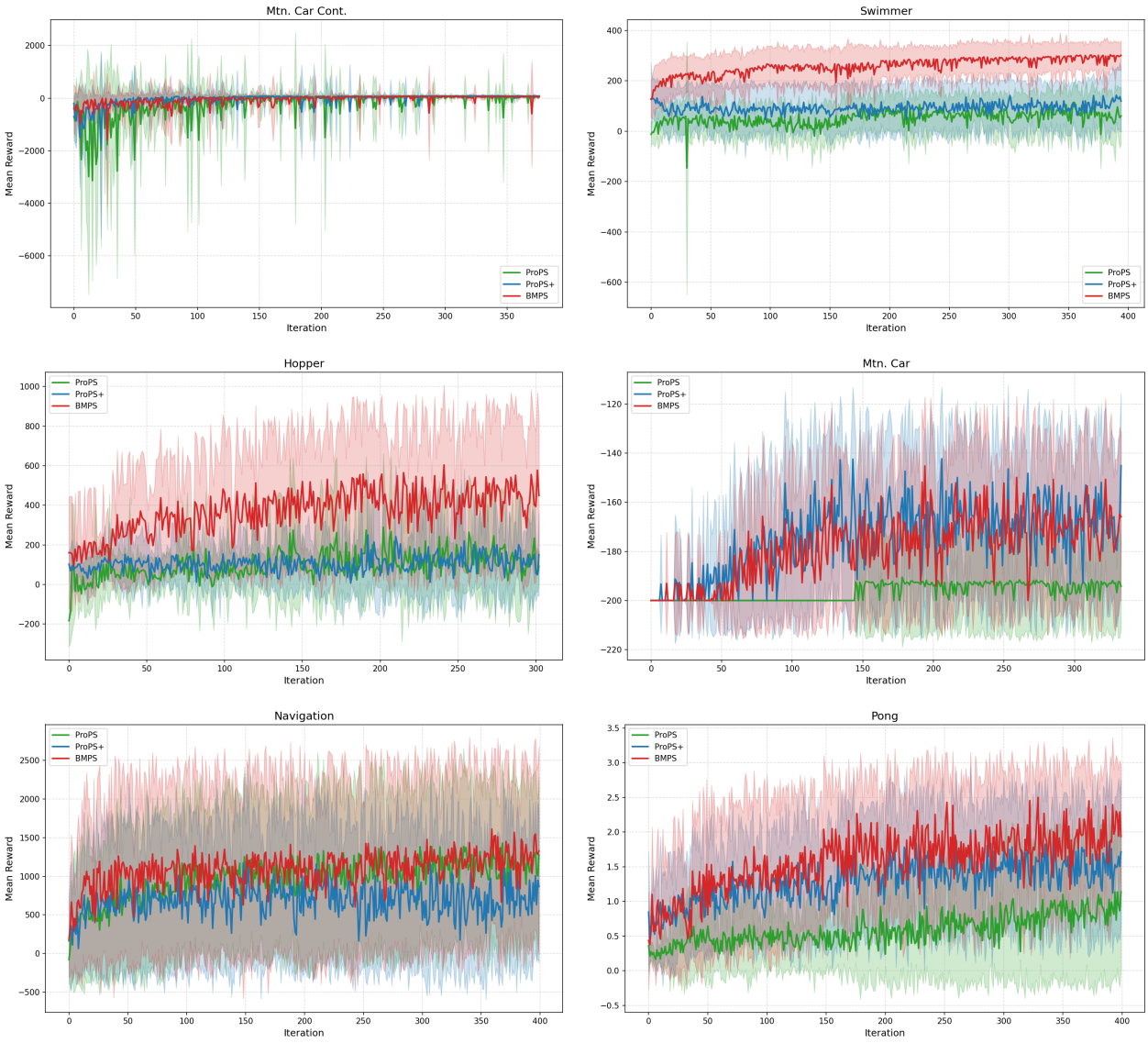


Figure F.1: Convergence comparison of LLM-based methods on six representative tasks. Every method runs for 400 LLM search iterations, i.e., 400 LLM proposals per run, or until the prompt exceeds the LLM context window; curves that end before iteration 400 correspond to runs that reached the context window first. The figure reports the mean return of the proposal at each iteration (shaded: ± one standard deviation over runs) for BMPS and prior LLM-based baselines across continuous-control tasks (MountainCarContinuous, Swimmer, Hopper) and discrete or navigation-style tasks (MountainCar, Navigation, Pong).

Figure F.1 compares the search trajectories of BMPS, ProPS, and ProPS+ on six tasks, complementing the final-return tables. The curves illustrate task-dependent progress, but do not by themselves establish a general advantage in convergence speed or stability, or isolate the contribution of visual feedback. The final-return comparisons in Appendix F.3 provide the statistical evidence; on Hopper and Navigation, BMPS has a higher mean than the LLM-based baselines, but those differences are not significant after correction.

F.3 Statistical Significance

We test whether the differences in Tables 1 and 2 are statistically significant. *Per environment*, we compare BMPS with each baseline using a two-sided Welch's t-test over the 10 runs, computed from per-run returns when available and otherwise from the reported means and standard deviations, and apply the Holm–Bonferroni correction [5] across baselines within each environment at $\alpha = 0.05$. *Across environments*, we rank the ten methods evaluated on all 13 environments, apply the Friedman test [2], and compare BMPS with each baseline using a

Table F.2: Significance of BMPS against each baseline. W/T/L counts the environments in which BMPS is significantly better, not significantly different, or significantly worse (two-sided Welch's t-test, Holm-corrected, $\alpha = 0.05$). Avg. rank is over the 13 environments (BMPS: 1.92); the last column gives Holm-corrected one-sided Wilcoxon signed-rank p-values across environments. DQN applies only to the six discrete tasks.

Baseline	W/T/L	Avg. rank	Wilcoxon p
ProPS+	3/10/0	3.38	0.0020
ProPS	6/7/0	4.31	0.0020
TRPO	6/6/1	4.42	0.0378
PPO	7/5/1	5.54	0.0121
CMA-ES	8/4/1	5.92	0.0121
(μ, λ)-ES	10/3/0	6.31	0.0043
A2C	9/3/1	7.31	0.0098
OpenAI-ES	9/4/0	7.58	0.0020
ARS	10/3/0	8.31	0.0011
DQN	5/1/0	–	–

Table F.3: Holm-adjusted p-values of two-sided Welch's t-tests comparing BMPS with each baseline per environment (10 runs per method; Holm correction across baselines within each environment). Bold entries are significant at $\alpha = 0.05$; $^+$ marks BMPS better and $^-$ marks BMPS worse. When both methods have zero variance, we use the exact Mann–Whitney p-value for complete separation.

Environment	A2C	DQN	PPO	TRPO	ProPS	ProPS+	OpenAI-ES	CMA-ES	(μ, λ)-ES	ARS
Mtn. Car Cont.	**<.001**$^+$	–	**<.001**$^+$	**<.001**$^+$	.553	.176	**<.001**$^+$	**.005**$^+$	**.011**$^+$	**<.001**$^+$
Inv. Pendulum	**<.001**$^+$	–	**<.001**$^+$	**.029**$^+$	.337	.337	**.008**$^+$	.110	**.029**$^+$	**.001**$^+$
Inv. Dbl. Pend.	1.000	–	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Reacher	**.001**$^+$	–	.884	.064	.357	.663	.357	**.001**$^+$	.617	.357
Swimmer	**<.001**$^+$	–	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	**.030**$^+$	**.001**$^+$	.159	**.043**$^+$	**<.001**$^+$
Hopper	**.003**$^+$	–	.070	.939	.180	.180	.177	**.007**$^-$	.054	.075
Walker	.065	–	.623	.623	**.007**$^+$	**.015**$^+$	**<.001**$^+$	**.015**$^+$	**<.001**$^+$	**<.001**$^+$
Frozen Lake	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$
Cliff Walking	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	.658	**<.001**$^+$	.420	.420	.126	**<.001**$^+$	**.019**$^+$
Maze	1.000	.839	**<.001**$^+$	1.000	1.000	1.000	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$
Mtn. Car	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	.530	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$	**<.001**$^+$
Navigation	**.031**$^-$	**.002**$^+$	**.006**$^-$	**.005**$^-$	.949	.949	**.003**$^+$	**.034**$^+$	**.034**$^+$	**.002**$^+$
Pong	**<.001**$^+$	**<.001**$^+$	.113	**.005**$^+$	**.030**$^+$	.113	**<.001**$^+$	**.001**$^+$	**<.001**$^+$	**<.001**$^+$

one-sided Wilcoxon signed-rank test [9] on per-environment mean returns min–max normalized across methods, again with the Holm correction.

Table F.2 summarizes the results, and Table F.3 reports all per-environment p-values. The Friedman test rejects the hypothesis that all methods perform equally ($\chi^2(9) = 53.38$, $p < 10^{-7}$), and BMPS attains the best average rank (1.92, followed by ProPS+ at 3.38). With the Nemenyi critical difference of 3.76, BMPS ranks significantly better than CMA-ES, (μ, λ)-ES, A2C, OpenAI-ES, and ARS. The across-environment Wilcoxon tests favor BMPS against each of the nine baselines evaluated on all 13 tasks (Holm-corrected $p \leq 0.05$; $p = 0.038$ for TRPO); this aggregate result does not imply superiority on every task. Per environment, BMPS is significantly worse than CMA-ES on Hopper and than A2C, PPO, and TRPO on Navigation. No significant deficit is detected against ProPS or ProPS+. Against ProPS+, BMPS is significantly better on Swimmer, Walker, and Frozen Lake; the other ten comparisons do not reach significance, which does not establish equivalence. BMPS has the higher mean on 12 tasks and ties on Maze, with the across-environment comparison yielding $p = 0.002$.

For the per-environment observations in Sections 5.2 and 5.3, the corrected tests give the following. The deficit of BMPS against CMA-ES on Hopper is significant ($p = 0.007$), whereas its lower mean than TRPO on Inverted Double Pendulum is not significant amid the large run-to-run variability. On Navigation, BMPS is significantly worse than A2C, PPO, and TRPO. On Frozen Lake, BMPS is significantly better than every baseline. On Mountain Car, its higher mean than ProPS+ is not significant ($p = 0.530$). Per-task tests thus identify gains over ProPS+ on Swimmer, Walker, and Frozen Lake, alongside deficits against CMA-ES on Hopper and policy-gradient methods on Navigation.

F.4 Robustness of Per-Environment Comparisons

Figure F.2 complements Table F.3 on five representative environments, chosen to include both environments where BMPS wins (Swimmer, Walker, Frozen Lake) and environments where it is beaten by some baseline (Hopper, Navigation). For each method we show the mean return with a 95% confidence interval $\bar{x} \pm t_{0.975,9}\, s/\sqrt{10}$, where s is the sample standard deviation over 10 runs. The standard deviations in Tables 1 and 2 are population standard deviations σ, and we convert them by $s = \sigma\sqrt{10/9}$. Because per-run returns are not available for every baseline, intervals are computed from the reported means and standard deviations rather than drawn as box plots. Next to each baseline we print the exact Holm-adjusted p-value of the two-sided Welch's t-test against BMPS.

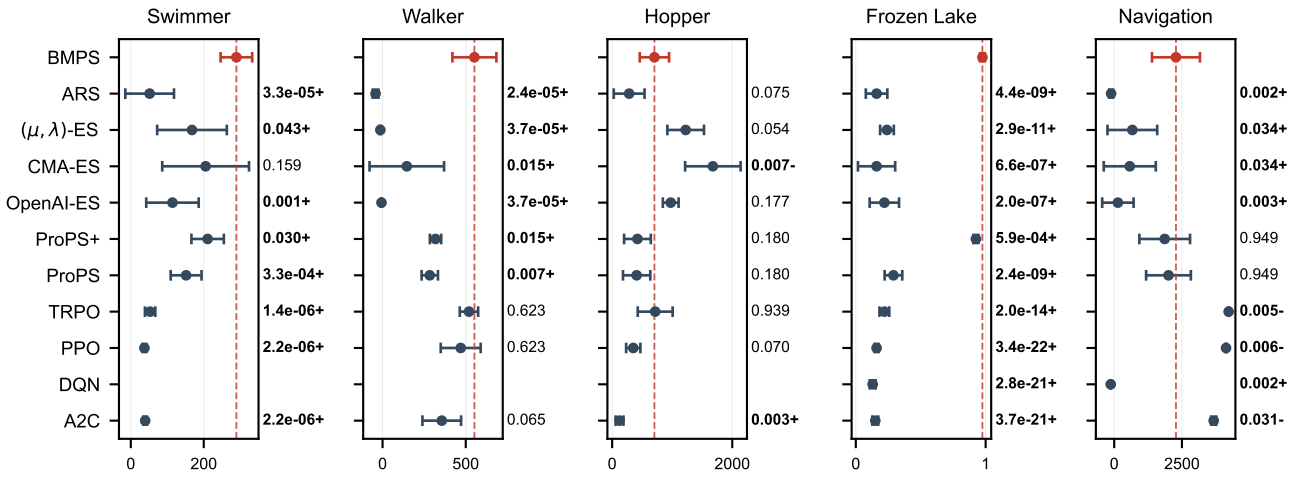


Figure F.2: Mean return and 95% confidence interval (10 runs) of each method on five representative environments. The dashed red line marks the BMPS mean. Numbers on the right are exact Holm-adjusted p-values of two-sided Welch's t-tests against BMPS; bold entries are significant at $\alpha = 0.05$, with + marking BMPS better and − marking BMPS worse. DQN is evaluated only on discrete environments.

Table F.4 reports 95% confidence intervals for BMPS and the two LLM-based baselines on all 13 environments. Intervals are computed from the 10 per-run returns where these are available, and otherwise from the reported mean and standard deviation (marked †).

Table F.4: Mean return and 95% confidence interval $[\bar{x} - t_{0.975,9}\, s/\sqrt{10},\ \bar{x} + t_{0.975,9}\, s/\sqrt{10}]$ over 10 runs, where $s = \sqrt{\frac{1}{n-1}\sum_i (x_i - \bar{x})^2}$ is the sample standard deviation. Tables 1 and 2 report the population standard deviation σ, so $s = \sigma\sqrt{n/(n-1)}$ with $n = 10$. †Computed from the reported mean and standard deviation because per-run returns are unavailable.

Environment	BMPS	ProPS+	ProPS
Mtn. Car Cont.	99.13 [99.05, 99.21]	97.92 [96.49, 99.35]	99.09 [98.95, 99.23]†
Inv. Pendulum	1000.00 [1000.00, 1000.00]	882.19 [702.62, 1061.75]	819.17 [545.92, 1092.42]†
Inv. Dbl. Pend.	982.31 [−889.50, 2854.13]	144.52 [91.81, 197.23]	125.89 [101.26, 150.52]†
Reacher	−7.358 [−7.859, −6.857]	−7.607 [−7.870, −7.343]	−7.950 [−8.387, −7.513]†
Swimmer	289.47 [246.20, 332.74]	210.65 [166.24, 255.06]	151.64 [109.39, 193.89]†
Hopper	703.97 [459.84, 948.10]	421.35 [198.96, 643.74]†	407.86 [180.63, 635.09]†
Walker	552.02 [419.77, 684.28]	318.46 [284.47, 352.45]	283.88 [234.74, 333.02]†
Frozen Lake	0.975 [0.956, 0.994]	0.925 [0.905, 0.945]	0.290 [0.222, 0.358]†
Cliff Walking	−64.35 [−68.86, −59.84]	−69.73 [−76.14, −63.32]	−100.00 [−100.00, −100.00]†
Maze	0.970 [0.970, 0.970]	0.970 [0.970, 0.970]†	0.970 [0.970, 0.970]†
Mtn. Car	−112.37 [−114.31, −110.42]	−117.47 [−135.08, −99.86]	−190.58 [−211.90, −169.26]†
Navigation	2279.5 [1395.2, 3163.8]	1863.7 [928.6, 2798.9]	2003.4 [1178.1, 2828.7]†
Pong	2.955 [2.875, 3.035]	2.650 [2.326, 2.974]†	1.890 [1.151, 2.629]†

G Detailed Algorithm

Algorithm G.1 gives a step-by-step description of the full BMPS pipeline. Notation follows Section 3.

Algorithm G.1 Behavior-Guided Multimodal Policy Search

Require: Environment $\mathcal{E}$, task description c, LLM, VLM, grid step ϵ, Poisson rate λ, neighbors per anchor N, evaluation rollouts L, warm-up size W, iterations T
Ensure: Policy with the highest recorded return

// Warm-up phase
Sample W random policies $\theta_1, \ldots, \theta_W \in \Theta$
$\mathcal{H}_0 \leftarrow \{(\theta_k, \widehat{J}(\theta_k))\}_{k=1}^{W}$ ▷ $\widehat{J}$: mean return over L rollouts
$\theta_{\text{cur}} \leftarrow \theta_W$ ▷ Last warm-up policy

// Main optimization loop
for $t = 1$ to T **do**
  // Step 1: Anchor selection
  $\theta_{\text{best}} \leftarrow$ policy with the highest recorded return in $\mathcal{H}_{t-1}$
  $\theta_{\text{worst}} \leftarrow$ policy with the lowest recorded return in $\mathcal{H}_{t-1}$
  $\mathcal{A}_t \leftarrow \{\text{cur}, \text{best}\}$; add worst if its recorded return differs from that of best

  // Step 2: Neighborhood sampling
  for $a \in \mathcal{A}_t$ **do**
    $\theta_{a,0} \leftarrow \theta_a$
    for $j = 1$ to N **do**
      $s_j \sim \text{Poisson}(\lambda)^{\otimes d}, \quad \sigma_j \sim \text{Uniform}(\{-1, +1\})^{\otimes d}$
      $\theta_{a,j} \leftarrow \text{clip}_{[-6,6]}\big(\text{round}(\theta_a + \epsilon\, \sigma_j \odot s_j)\big)$ ▷ Section 3.2
    end for
    for $j = 0$ to N **do**
      Run one rollout $\tau_{a,j}$ of $\pi_{\theta_{a,j}}$ in $\mathcal{E}$; $\quad R_{a,j} \leftarrow R(\tau_{a,j})$
    end for
  end for

  // Step 3: Multimodal analysis (VLM calls issued in parallel)
  for each $a \in \mathcal{A}_t$ and $j \in \{0, \ldots, N\}$ **do**
    $F(\tau_{a,j}) \leftarrow$ first frame, one frame every 50 steps, and final frame
    $b_{a,j} \leftarrow \text{VLM}\big(F(\tau_{a,j}), c_{a,j}\big)$ ▷ Behavioral description
  end for
  $\psi_t \leftarrow \text{Format}\{(\theta_{a,j}, R_{a,j}, b_{a,j})\}$ ▷ Neighbors sorted by return

  // Step 4: Language-guided policy proposal
  $\theta_t \sim P_{\text{LLM}}(\cdot \mid \mathcal{H}_{t-1}, c, \psi_t)$ ▷ Semantics-aware mutation

  // Step 5: Evaluation and history update
  $\mathcal{H}_t \leftarrow \mathcal{H}_{t-1} \cup \{(\theta_t, \widehat{J}(\theta_t))\}$
  $\theta_{\text{cur}} \leftarrow \theta_t$
end for
return policy with the highest recorded return in $\mathcal{H}_T$

H Environment Descriptions

In this section, we briefly describe the environments used in our experiments, with emphasis on the optimization challenges they present for black-box policy search. We include both continuous-control and discrete-control tasks to illustrate the breadth of settings in which BMPS is evaluated.

H.1 Discrete-Control Tasks

FrozenLake-v1. FrozenLake is a slippery grid-world task in which the agent must navigate from a start cell to a goal cell while avoiding holes. The observation space is `Discrete(16)`, corresponding to the agent's location on a 4×4 grid, and the action space is `Discrete(4)` with actions {left, down, right, up}. The reward is sparse: the agent receives +1 only when it reaches the goal and 0 otherwise. Episodes terminate when the goal is reached, the agent falls into a hole, or the time limit is reached. This environment is challenging for black-box search because useful reward feedback is extremely sparse and the default slippery dynamics make action outcomes stochastic.

Cliff Walking-v1. CliffWalking is a discrete navigation task on a 4×12 grid in which the agent must move from a start state to a goal state while avoiding a cliff region. The observation space is `Discrete(48)` and the action space is `Discrete(4)`. The agent receives a reward of -1 per step and a large penalty when stepping into the cliff, after which it is reset to the start. Reaching the goal ends the episode. In the standard Gymnasium version used here, the environment is not slippery by default, so the challenge comes primarily from the severe penalty structure rather than stochastic transitions. This task is therefore representative of deceptive landscapes, where locally reasonable behavior can still incur catastrophic penalties.

MountainCar-v0. MountainCar is a classic underpowered control problem in which the agent must drive a car up a hill despite insufficient direct engine power. The observation space is `Box(2,)` containing position and velocity, and the action space is `Discrete(3)` for accelerating left, applying no force, or accelerating right. The agent receives a reward of -1 at each time step until the goal is reached. Episodes terminate when the car reaches the goal position or the time horizon is exhausted. This task is difficult because good behavior requires coordinated long-horizon momentum building before any clear improvement in return is observed.

Acrobot-v1. Acrobot is a two-link pendulum system in which only the second joint is actuated, and the objective is to swing the end effector upward to a target height. The observation space is continuous, typically represented by trigonometric encodings of joint angles together

with angular velocities, while the action space is `Discrete(3)`. The agent incurs a negative per-step reward until the target configuration is reached or the episode ends. The task is challenging because effective control depends on exploiting nonlinear dynamics rather than making locally greedy moves.

LunarLander-v3. LunarLander is a discrete-action landing task in which the agent must control a spacecraft to land softly on a designated pad. The observation space is continuous and includes position, velocity, orientation, angular velocity, and leg-contact indicators, while the action space is `Discrete(4)`. The reward combines shaping terms for stable descent with penalties for crash-like behavior and bonuses for successful landing. Episodes terminate upon landing, crashing, or reaching the step limit. This environment is useful for evaluating whether black-box search can handle multi-objective feedback with both dense shaping and sparse success events.

Navigation. Navigation is a custom 2D driving-style task in which the agent moves along a track while avoiding collisions with the border. The observation space is `Box(5,)` corresponding to five lidar-style distance measurements, and the action space is `Discrete(3)` for moving forward, rotating left, or rotating right. Rewards encourage forward progress, mildly penalize turning, and assign a large negative penalty upon collision. Episodes terminate when the agent hits the border or reaches the maximum horizon. The task highlights the role of structured behavioral feedback because safe progress depends strongly on spatial interpretation of the scene.

Pong. Pong is a custom discrete-action paddle task in which the agent controls a paddle on one side while facing a rule-based opponent. The observation space is `Box(5,)` and encodes the ball state together with the agent paddle position, while the action space is `Discrete(3)` for moving up, staying still, or moving down. The agent receives positive reward when it successfully returns the ball. Episodes terminate when the agent misses the ball or reaches the hit limit. This environment tests whether policy search can capture reactive timing behavior from compact state descriptions.

H.2 Continuous-Control Tasks

MountainCarContinuous-v0. MountainCarContinuous is the continuous-action version of MountainCar. The observation space remains `Box(2,)` with position and velocity, but the action space becomes `Box(1,)` with a continuous force input. The reward provides a large bonus for reaching the goal while penalizing large actions through a quadratic control cost. Episodes terminate when the goal is reached or the time limit is exceeded. Compared with the discrete version, this task additionally requires choosing an appropriate magnitude of force, making fine-grained policy updates more important.

Swimmer-v5. Swimmer is a MuJoCo locomotion task featuring a planar three-link robot that must learn to move forward through joint actuation. The observation space is continuous and contains joint angles and velocities, while the action space is `Box(2,)` corresponding to torques on the two actuated joints. The reward is the forward velocity term minus a control cost. Episodes terminate after a fixed horizon. Swimmer is a useful benchmark for black-box optimization because its reward is dense but the mapping from local actuation to efficient long-term locomotion is still nontrivial.

Hopper-v5. Hopper is a MuJoCo task in which a one-legged robot must hop forward without falling. The observation space contains body positions and velocities, and the action space is `Box(3,)` over joint torques. The reward combines forward progress, a survival bonus, and a control penalty. Episodes terminate when the robot falls or violates stability constraints, or when the maximum horizon is reached. The task is sensitive to unstable policies, making it a good test of whether BMPS can distinguish productive motion from visually plausible but fragile behavior.

InvertedPendulum-v5. InvertedPendulum is a continuous-action balancing task in which a cart must keep a pendulum upright. The observation space is `Box(4,)` and includes cart position, pole angle, and their velocities, while the action space is `Box(1,)` corresponding to the applied force. The reward is typically +1 for each time step the pendulum remains upright. Episodes terminate when the pendulum falls or the time limit is reached. This task provides a relatively clean benchmark for studying whether BMPS can exploit trajectory-level diagnostics even in simpler stabilizing control problems.

InvertedDoublePendulum-v5. InvertedDoublePendulum extends the previous task to a two-link system with substantially more complex dynamics. The observation space is continuous and higher-dimensional, and the action space is again `Box(1,)`. The reward includes an alive bonus together with penalties for deviation from the upright configuration and excessive velocity. Episodes terminate when the system collapses or when the step limit is reached. The task is significantly harder because maintaining stability requires coordinated control of multiple coupled bodies.

Reacher-v5. Reacher is a two-link robotic arm task in which the objective is to move the fingertip toward a target. The observation space is `Box(11,)` and includes arm configuration, velocities, and target-relative coordinates, while the action space is `Box(2,)` for joint torques. The reward is the negative sum of distance-to-target and control effort. Episodes terminate after a short horizon. This environment is valuable because performance depends on precise geometry and smooth coordinated movement rather than simple forward locomotion.

Walker2d-v5. Walker2d is a MuJoCo locomotion task with a planar bipedal robot that must learn to walk forward while maintaining balance. The observation space contains body positions and velocities, and the action space is `Box(6,)`. The reward combines forward

progress, an alive bonus, and a control penalty, and episodes terminate when the walker falls or violates state constraints. Walker2d is more difficult than Swimmer or Hopper because stable forward motion requires coordination across a larger number of joints.

References

[1] Thomas M. Cover and Joy A. Thomas. 2006. *Elements of Information Theory* (2nd ed.). Wiley-Interscience, Hoboken, NJ, USA.

[2] Milton Friedman. 1937. The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance. *J. Amer. Statist. Assoc.* 32, 200 (1937), 675–701.

[3] Nikolaus Hansen. 2023. The CMA Evolution Strategy: A Tutorial. arXiv:1604.00772 [cs.LG] https://arxiv.org/abs/1604.00772

[4] Wassily Hoeffding. 1963. Probability Inequalities for Sums of Bounded Random Variables. *J. Amer. Statist. Assoc.* 58, 301 (1963), 13–30. doi:10.1080/01621459.1963.10500830

[5] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[6] Horia Mania, Aurelia Guy, and Benjamin Recht. 2018. Simple random search of static linear policies is competitive for reinforcement learning. In *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.), Vol. 31. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2018/file/7634ea65a4e6d9041cfd3f7de18e334a-Paper.pdf

[7] Daniel Russo and Benjamin Van Roy. 2016. An Information-Theoretic Analysis of Thompson Sampling. *Journal of Machine Learning Research* 17, 68 (2016), 1–30.

[8] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. 2017. Evolution Strategies as a Scalable Alternative to Reinforcement Learning. arXiv:1703.03864 [stat.ML] https://arxiv.org/abs/1703.03864

[9] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945), 80–83.